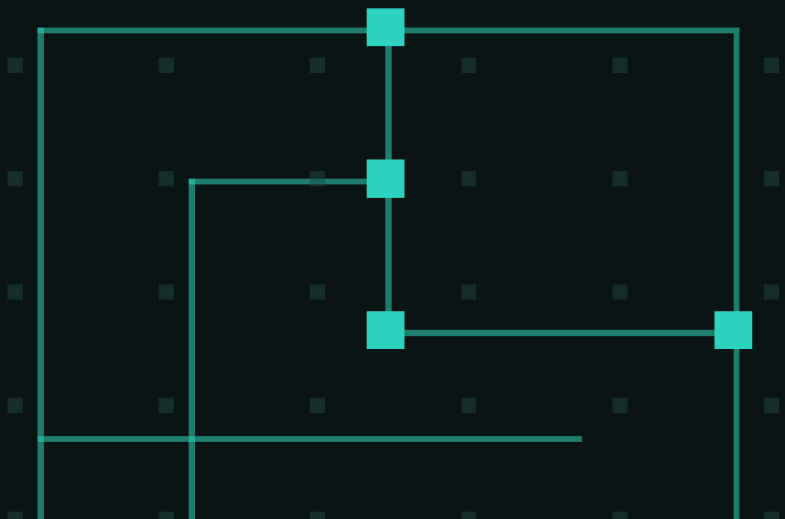
How Computers Work

A Practical Guide

- See inside the machine, from bits to the boot
- Understand memory, files, programs, and the OS
- Build the base for reversing, forensics, and exploitation





TOUCH DEN

Contents

Section 1: What a Computer Does 1

1. The Idea of a Computer 2
2. Input, Processing, Output, and Storage 3
3. Hardware and Software 3
4. Everything Inside Is Numbers 4
5. Instructions and Data 5
6. Why Security People Must Understand the Machine 5
7. A Map of This Course 6

Section 2: Binary, Hexadecimal, and Bits 9

1. Bits and Bytes 10
2. Counting in Binary 11
3. Converting Between Binary and Decimal 11
4. Hexadecimal, and Why It Is Used 12
5. Units: Kilobytes, Megabytes, Bits vs. Bytes 13
6. Bitwise Thinking: AND, OR, XOR, and NOT 13
7. How Negative Numbers Are Stored 14
8. Reading a Hex Dump 14

Section 3: Representing Text: ASCII, Unicode, and Base64 18

1. How Text Becomes Numbers 19
2. ASCII 20
3. The Problem of Other Languages 20
4. Unicode and UTF-8 21
5. Khmer and Other Scripts in Unicode 21
6. Base64: Encoding, Not Encryption 22
7. URL and Percent Encoding 23
8. Why Encoding Matters in Security 23

Section 4: The CPU, Memory, and Storage 27

1. The CPU and the Fetch–Execute Cycle 28
2. Registers 29
3. RAM: Fast and Temporary 29
4. Storage: Slower and Permanent 30
5. The Memory Hierarchy and Cache 31
6. Clock Speed, Cores, and Threads 31
7. Why "Volatile" Matters (a Forensics Preview) 32

Section 5: What an Operating System Does 35

1. The OS as the Manager 36
2. The Kernel 37
3. User Space and Kernel Space 37
4. System Calls 38
5. Drivers and Hardware 38
6. Multitasking 39
7. Users, Permissions, and the OS as a Security Boundary 39

Section 6: Processes, Threads, and Memory 43

1. What a Process Is 44
2. How a Process Is Laid Out in Memory 45
3. The Stack and the Heap 45
4. Threads 46
5. Virtual Memory and Isolation 47
6. How the OS Keeps Processes Apart 47
7. Looking at Running Processes 48

Section 7: Files and File Systems 52

1. What a File Really Is 53

2. File Systems 54
3. Paths and Directories 54
4. Metadata and Permissions 55
5. File Types and Magic Bytes, and Why the Extension Can Lie 55
6. Deleting vs. Really Deleting 56
7. Mounting and Devices 57

Section 8: How Programs Are Built and Run 60

1. From Source Code to a Running Program 61
2. Compiled vs. Interpreted Languages 62
3. The Build Pipeline: Compile and Link 63
4. Machine Code and Assembly 63
5. Libraries: Static and Dynamic 64
6. What an Executable File Is (ELF, PE, Mach-O) 65
7. Why This Matters for Reversing and Malware 66

Section 9: The Boot Process 69

1. From Power Button to Desktop 70
2. Firmware: BIOS and UEFI 71
3. The Bootloader 71
4. Loading the Kernel 72
5. Init and the First Processes 72
6. Where Malware Hides at Boot 73
7. Secure Boot 74

Section 10: Virtual Machines and Containers 77

1. What Virtualization Is 78
2. Hypervisors and Virtual Machines 79
3. Containers, and How They Differ From VMs 79
4. Why Security Work Relies on Both 80
5. Snapshots and Clean States 81
6. The Limits of Isolation 81

Section 11: A First Look at C and Memory 85

1. Why Security People Learn a Little C 86
2. A Tiny C Program 87
3. Variables Are Places in Memory 88
4. Pointers, Gently 88
5. The Stack and a Buffer 89
6. What a Buffer Overflow Is (a Preview) 90
7. Memory Safety, and Why It Matters 91

Section 12: Explore a Running System 94

1. The Goal: See the Ideas for Real 95
2. Who Am I, and What Is Running 96
3. Identifying and Reading Files 96
4. How Long Up, and How It Started 97
5. Tying It Together 98

References 102

How to Use This Book

This is the second course in a cyber security program. It assumes only the first course, Security Fundamentals; no programming and no prior knowledge of how computers work inside. It has 12 sections, each building on the ones before, so read them in order.

Every section follows the same plan: **What You Will Learn** and **Before You Start**; numbered **topics**, one idea each; **Common Mistakes**; and **Summary**, **Check Your Understanding**, and **Practice** with answers and worked examples.

This course has a little code and commands. Later sections show small C programs and short shell commands (`xxd` , `file` , `ps` , `uptime`) to let you *see* memory, files, and processes for real. Everything is explained from zero and runs on your own computer. You need no programming to read it; to try the commands, a terminal helps.

Run things on your own machine only. The one rule from Security Fundamentals applies throughout: only look at systems you own or have written permission to test. Every example here is for understanding and defending; nothing targets a system that isn't yours. Keys, prices, and names in examples are made up.

The through-line. This course is the trunk the later courses grow from: the stack and buffers lead to Binary Exploitation; executables to Reverse Engineering and Malware Analysis; volatile memory and deleted files to Digital Forensics; processes and the kernel to everything. Each idea is introduced lightly here and names the course that goes deep.

Course mark icon from Material Symbols by Google, Apache License 2.0.

1

Section 1

What a Computer Does

IN THIS SECTION

1. The Idea of a Computer 2

2. Input, Processing, Output, and Storage 3

3. Hardware and Software 3

4. Everything Inside Is Numbers 4

5. Instructions and Data 5

6. Why Security People Must Understand the Machine 5

7. A Map of This Course 6

What You Will Learn

- What a computer really is, under the surface.
- The four things every computer does: input, processing, output, and storage.
- The difference between hardware and software.
- Why everything inside a computer is just numbers.
- Why instructions and data are both stored the same way, and why that matters.
- Why a security person has to understand the machine.
- A map of this course.

Before You Start

You need only Security Fundamentals (course 01) and curiosity. No programming, and no prior knowledge of how computers work inside. This course has a little code and a few commands in later lessons, but this first lesson is all ideas.

Why this course exists: in Security Fundamentals you learned *what* to protect and *why*. To actually attack or defend a system, you must understand the machine it runs on. A reverse engineer reads a program's instructions; a forensic analyst recovers data from memory and disks; an exploit developer bends a program's use of memory. All of it rests on the handful of ideas in this course.

---

1. The Idea of a Computer

A computer looks complicated, but the core idea is simple: **a computer is a machine that follows a list of instructions, step by step, very fast, to work with information.**

That is all a computer fundamentally does. It does not think or understand. It follows instructions, one after another, billions of times a second, without ever getting bored or making a different choice than the instructions say. Everything a computer appears to "do", show a video, run a game, send a message, is really a vast number of tiny, simple steps carried out so quickly that the result looks instant and alive.

Two things make a computer powerful:

- **Speed:** a modern processor carries out billions of simple steps every second.
- **Reliability:** it does exactly what the instructions say, every time, the same way.

This is also why computers are a security problem. A computer will follow *whatever* instructions it is given, including an attacker's, if the attacker can get their instructions to run. Much of security is about controlling which instructions a machine will follow, and with whose authority.

2. Input, Processing, Output, and Storage

Every computer, from a phone to a giant server, does four things. Keep these four words; the whole course hangs on them.

Stage	What it means	Everyday example
Input	Information comes in	Typing, tapping, a network message, a sensor
Processing	The computer works on it, following instructions	Calculating, sorting, deciding
Output	A result goes out	The screen, a sound, a network reply
Storage	Information is kept for later	Saving a file, remembering a setting

A simple example: you type a search (input), the computer looks through its data and ranks results (processing), it shows them on the screen (output), and it remembers your recent searches (storage).

Every part of a computer serves one of these four jobs, and so does every security concern. An attacker might poison the **input** (a malicious message), hijack the **processing** (run their own instructions), read the **output** (spy on your screen), or steal the **storage** (copy your files). When you meet a new attack later in the program, it helps to ask: which of the four is it attacking?

3. Hardware and Software

A computer has two sides that work together.

- **Hardware** is the physical machine: the parts you could touch. The processor, the memory chips, the disk, the screen, the keyboard, the network card.
- **Software** is the instructions: the programs that tell the hardware what to do. An operating system, an app, a game, a virus, are all software.

The relationship is simple: **software is a list of instructions, and hardware is the machine that carries them out.** Hardware without software is an inert lump; software without hardware is just a list that does nothing. A program only has an effect when hardware runs it.

Between the two sits a special piece of software called the **operating system** (Windows, macOS, Linux, Android), which manages the hardware and lets other programs use it. Lesson 5 is all about it. For now, picture three layers: the hardware at the bottom, the operating system in the middle managing it, and your apps on top.

This split matters for security because attacks happen at every layer. Some target hardware, most target software, and the most serious often target the operating system in the middle, because whoever controls it controls everything above.

4. Everything Inside Is Numbers

Here is the idea that surprises people most, and that unlocks the rest of the course: **inside a computer, everything is numbers.** Not just sums and prices, but *everything*.

- A letter of text is a number (lesson 3).
- A color in a photo is a set of numbers.
- A sound is a long list of numbers.
- A program's instructions are numbers (lesson 8).
- An image, a video, a document, a web page, all numbers.

A computer has no idea what a number "means". The same number could be a letter, a brightness, or an instruction, depending only on how the computer is told to treat it. This is a deep and powerful fact. It means one kind of machine, working only with numbers, can handle every kind of information in the world.

And the numbers themselves are stored in the simplest possible way: as huge quantities of **on-or-off** switches. Everything, every photo and song and program, is ultimately a pattern of ons and offs. The next lesson shows how that works, with **bits** and **binary**. For now, hold the headline: to a computer, there is no text, no picture, no program, only numbers, stored as switches.

This is why topics like encoding (lesson 3) and file formats (lesson 7) matter so much in security. If everything is numbers, then the *rules* for reading those numbers are everything, and attackers love to break or confuse those rules.

5. Instructions and Data

Since everything is numbers, a computer stores two different kinds of thing in the very same way:

- **Data:** the information a program works on, a photo, a message, a list of prices.
- **Instructions:** the steps the program follows, the program itself.

Both are just numbers in memory. This idea, that a computer holds its instructions and its data together in the same memory, is called the **stored-program** design, and almost every computer ever built works this way. It is why a computer is so flexible: to make it do something new, you do not rebuild the machine, you just give it a new set of instructions (a new program).

But it has a dangerous side, which is the root of a whole class of attacks. If instructions and data live in the same memory and look the same, then a computer can be **tricked into treating data as instructions.** An attacker sends what looks like ordinary data, a message, a file, an image, but arranges it so the computer runs part of it as instructions. That is the essence of many serious attacks (buffer overflows, code injection), which you will study in later courses. It all starts here: instructions and data are made of the same stuff.

6. Why Security People Must Understand the Machine

You might wonder why a security course spends time on CPUs and memory. Because **you cannot secure, attack, or investigate what you do not understand.** Every area of security rests on the machine:

Security work	The machine knowledge it needs
Reverse engineering (course 17)	Reading a program's instructions and memory
Binary exploitation (course 18)	Bending how a program uses memory, especially the stack
Malware analysis (course 29)	Understanding executables, processes, and the OS
Digital forensics (course 28)	Recovering data from memory, disks, and deleted files
Defense and detection	Knowing what "normal" processes, files, and boot look like

Notice how often the later courses appear. This course is the trunk; those are the branches. When a forensic analyst recovers a deleted file, they are using what lesson 7 teaches about how files are stored. When an exploit developer overflows a buffer, they are using what lessons 6 and 11 teach about the stack. Skip the trunk, and the branches never hold.

There is also a simpler reason: understanding the machine removes fear. A computer is not magic. It is a fast, literal follower of numeric instructions. Once you see that clearly, security stops being mysterious and becomes something you can reason about.

7. A Map of This Course

Here is where we are going, and why the order makes sense.

Part	Lessons	What you will understand
Numbers and text	2–3	How numbers (binary, hex) and text (ASCII, Unicode, Base64) are stored
The machine	4–7	The CPU, memory, and storage; the operating system; processes; files
Programs and boot	8–10	How programs are built and run; how a computer starts; virtual machines
Memory, up close	11–12	A first look at C and memory, then a hands-on project on a real system

Each lesson builds on the one before. By the end, you will be able to open up a running computer, inspect its processes, read a file in raw form, follow it from power-on to desktop, and understand, in outline, how a program lives in memory. That is the foundation every later security course assumes.

One last framing, carried from Security Fundamentals: everything here is for understanding and defending. We look inside our *own* machines. Nothing in this course requires, or encourages, touching a computer that is not yours.

In the next lesson, we start at the very bottom, with the ons and offs themselves: bits, binary, and hexadecimal.

Common Mistakes

1. Thinking a computer "understands" what it does. It does not. It follows numeric instructions, literally and fast. That literalness is exactly what attackers exploit.

2. Believing software is separate from numbers. A program is numbers in memory, just like data. This is the root of many attacks.

3. Treating hardware and software as the same thing. Hardware is the machine; software is the instructions it runs. Attacks target different layers, so the distinction matters.

4. Assuming "it's just data, it's harmless." Because instructions and data share the same memory, carefully crafted data can be run as instructions. Input is never automatically safe.

5. Skipping the fundamentals to get to "the hacking." Reversing, exploitation, and forensics all rest on these basics. Without them, the advanced courses are memorized recipes, not understanding.

Summary

Idea	Remember
What a computer is	A machine that follows numeric instructions, step by step, very fast
The four jobs	Input, processing, output, storage
Hardware vs software	The physical machine vs the instructions it runs
Everything is numbers	Text, images, sound, and programs are all numbers, stored as on/off switches
Instructions and data	Both are numbers in the same memory (stored-program); this enables both flexibility and attacks
Why it matters	You cannot attack, defend, or investigate what you do not understand

Check Your Understanding

1. In one sentence, what is a computer, at its core?
2. Name the four things every computer does, with an example of each.
3. What is the difference between hardware and software?
4. "Inside a computer, everything is numbers." Give three different things that are stored as numbers.
5. What does the stored-program idea mean, and why does it make computers flexible?
6. How does the stored-program idea also enable a class of attacks?
7. Give two examples of security work and the machine knowledge each needs.

Answers

1. A computer is a machine that follows a list of instructions, step by step, very fast, to work with information.
2. Input (information comes in, e.g. typing), processing (working on it, e.g. calculating), output (a result goes out, e.g. the screen), storage (keeping it, e.g. saving a file).
3. Hardware is the physical machine (processor, memory, disk); software is the instructions (programs) that tell the hardware what to do.

4. Any three: text/letters, colors in a photo, sound, program instructions, images, video, documents.
5. A computer holds its instructions and data together in the same memory. It makes computers flexible because you give the machine new behavior just by giving it a new program, not by rebuilding it.
6. Because instructions and data are the same kind of thing in the same memory, a computer can be tricked into treating crafted data as instructions (e.g. buffer overflows, code injection).
7. Any two: reverse engineering needs reading a program's instructions/memory; forensics needs recovering data from memory and disks; exploitation needs understanding how programs use memory; malware analysis needs understanding executables, processes, and the OS.

Practice

No computer needed, just think and write.

1. Pick a everyday action on your phone (sending a photo, playing a song, paying with a QR code). Break it into the **four stages**: what is the input, the processing, the output, and the storage?
2. For that same action, name one thing that is stored as **numbers** even though it does not look like a number (for example, the photo's colors).
3. For that same action, name one of the four stages an **attacker** might target, and what they might gain.

A Worked Example

Paying with a QR code:

1. Input: the camera reads the QR code, and you enter your PIN. Processing: the app checks the code, talks to the bank, and confirms you have the money. Output: a "payment successful" screen and a receipt. Storage: the transaction is saved in your history and the bank's records.
2. The QR code itself is just numbers (the merchant's ID and the amount, encoded as a pattern of black and white squares, which are 1s and 0s).
3. An attacker might target the **input** by showing you a fake QR code that pays *them* instead of the shop, gaining your money. (Defending against exactly this is part of the Fraud and Banking Security course.)

2

Section 2

Binary, Hexadecimal, and Bits

IN THIS SECTION

1. Bits and Bytes 10

2. Counting in Binary 11

3. Converting Between Binary and Decimal 11

4. Hexadecimal, and Why It Is Used 12

5. Units: Kilobytes, Megabytes, Bits vs. Bytes 13

6. Bitwise Thinking: AND, OR, XOR, and NOT 13

7. How Negative Numbers Are Stored 14

8. Reading a Hex Dump 14

What You Will Learn

- What bits and bytes are.
- How to count in binary, and convert between binary and decimal.
- What hexadecimal is, and why security tools use it everywhere.
- Units: kilobytes, megabytes, and the difference between bits and bytes.
- Bitwise thinking: AND, OR, XOR, and NOT.
- How computers store negative numbers.
- How to read a hex dump.

Before You Start

Read lesson 1. You know that inside a computer everything is numbers, stored as on/off switches. This lesson opens up those switches. A little arithmetic is all you need; we build everything from zero. The console examples use Python and the `xxd` tool, and are shown exactly as they ran on a real machine.

---

1. Bits and Bytes

At the very bottom, a computer stores everything using switches that are either **off** or **on**. We write off as **0** and on as **1**. A single one of these switches is called a **bit** (short for "binary digit"). A bit is the smallest piece of information there is: one yes-or-no, one 0-or-1.

One bit alone cannot say much, only two things (0 or 1). So computers group bits together. Eight bits grouped together make one **byte**. A byte is the basic unit a computer works with: files, memory, and network data are all measured in bytes.

How much can one byte hold? Eight bits, each 0 or 1, give 2 × 2 × 2 × 2 × 2 × 2 × 2 × 2 = **256** different patterns, from `00000000` to `11111111`. So one byte can represent 256 different values, usually the numbers 0 to 255. That number, 256, and its top value, 255, will appear again and again.

Term	Meaning
Bit	One switch: 0 or 1
Byte	8 bits grouped together; 256 possible values (0–255)
Nibble	4 bits (half a byte); you will meet this with hex

Everything larger, a letter, a number, a pixel, a whole file, is just many bytes in a row.

2. Counting in Binary

The numbers you use every day are in **decimal** (base 10): ten digits, 0 through 9. When you run out of digits past 9, you add a new column: 10, 11, and so on. Each column is worth ten times the one to its right: ones, tens, hundreds, thousands.

Computers use **binary** (base 2): only two digits, 0 and 1. The idea is identical, but each column is worth **two** times the one to its right, not ten. The columns are 1, 2, 4, 8, 16, 32, 64, 128, each a doubling.

To read a binary number, add up the column values wherever there is a 1:

Binary	128	64	32	16	8	4	2	1	Decimal
`0000 1101`	0	0	0	0	1	1	0	1	8 + 4 + 1 = **13**
`1111 1111`	1	1	1	1	1	1	1	1	**255**
`1000 0000`	1	0	0	0	0	0	0	0	**128**

That is all binary is: the same counting you already know, with two digits instead of ten. Computers use it because a switch has exactly two states, off and on, which map perfectly onto 0 and 1.

3. Converting Between Binary and Decimal

You rarely convert by hand in real work, a tool does it, but doing a couple yourself makes binary click.

Binary to decimal: add the column values where there is a 1. `0000 1101` has 1s in the 8, 4, and 1 columns: 8 + 4 + 1 = 13.

Decimal to binary: find the biggest column value that fits, subtract, and repeat. For 13: the biggest power of two that fits is 8 (leaving 5), then 4 (leaving 1), then 1 (leaving 0). So the 8, 4, and 1 columns are 1: `0000 1101`.

Any tool can check you. Here it is in Python, shown exactly as it ran:

```
$ python3 -c 'print(bin(13), 13); print(int("1010", 2))'
0b1101 13
10
```

The `0b` prefix is how Python (and many tools) say "this is binary". So `0b1101` is binary 1101, which is 13 in decimal, and binary `1010` is 10. You do not need to memorize conversions; you need to understand what the columns mean, and know that every tool can convert for you.

4. Hexadecimal, and Why It Is Used

Binary is correct but hard for humans: `11111111` is eight characters for one byte, and easy to miscount. So security tools almost always show bytes in **hexadecimal** (base 16), usually just called **hex**.

Hex has sixteen digits. The first ten are 0–9, as usual. For the values ten to fifteen, it borrows letters: A = 10, B = 11, C = 12, D = 13, E = 14, F = 15.

Why sixteen? Because 16 is 2×2×2×2, so **one hex digit is exactly four bits** (a nibble), and **two hex digits are exactly one byte.** That neat fit is the whole reason hex is used: it is a short, exact way to write bytes.

Binary (4 bits)	Hex	Decimal
`0000`	0	0
`1010`	A	10
`1111`	F	15
`1111 1111`	FF	255

So one byte, which is `00000000` to `11111111` in binary, is just `00` to `FF` in hex, two characters, always. Hex is often written with a `0x` prefix, so `0xFF` means hex FF, which is 255. Check it in Python:

```
$ python3 -c 'print(hex(255), int("ff", 16))'
0xff 255
```

From now on, whenever you see something like `0x4D` or `4d 69 6e 61`, you are looking at bytes written in hex. Get comfortable with it; it is the everyday language of memory, files, and network data in security.

5. Units: Kilobytes, Megabytes, Bits vs. Bytes

Bytes add up fast, so we use bigger units. Each step is about a thousand times the last (technically 1024, which is 2 to the power of 10, but "about a thousand" is close enough to reason with):

Unit	About how many bytes	Rough example
Kilobyte (KB)	~1,000	A page of text
Megabyte (MB)	~1,000,000	A photo or a song
Gigabyte (GB)	~1,000,000,000	A movie; your phone's memory in GB
Terabyte (TB)	~1,000,000,000,000	A large hard drive

One trap catches everyone: **a bit and a byte are not the same, and the difference shows up in a familiar place.** A byte is 8 bits. By convention, storage (files, disks) is measured in **bytes**, written with a capital B (MB, GB), while network and internet speeds are measured in **bits**, written with a lowercase b (Mbps = megabits per second).

That is why a "100 Mbps" internet connection downloads a file at only about 12 megabytes per second, not 100: you divide by 8, because the speed is in bits and the file is in bytes. Knowing this saves confusion, and it matters when you reason about how long it takes to move data, for example when exfiltrating or capturing it in later courses.

6. Bitwise Thinking: AND, OR, XOR, and NOT

Because everything is bits, computers can work on bits directly, one position at a time. These **bitwise operations** are everywhere in security: in permissions, network masks (lesson on subnetting in the Networking course), simple ciphers, and low-level code.

Line up two bytes and compare each bit position:

- **AND** gives 1 only if **both** bits are 1. (Used to *check* or *clear* bits.)
- **OR** gives 1 if **either** bit is 1. (Used to *set* bits.)
- **XOR** (exclusive or) gives 1 if the bits are **different**. (Used in simple encryption and to flip bits.)
- **NOT** flips every bit: 0 becomes 1, 1 becomes 0.

Here they are on two 4-bit values, run for real:

```
$ python3 -c 'a,b=0b1100,0b1010
print(f"{a:04b} AND {b:04b} = {a&b:04b}")
print(f"{a:04b} OR  {b:04b} = {a|b:04b}")
print(f"{a:04b} XOR {b:04b} = {a^b:04b}")'
1100 AND 1010 = 1000
1100 OR  1010 = 1110
1100 XOR 1010 = 0110
```

Read the AND line: only the leftmost position has a 1 in *both* numbers, so only that position is 1 in the answer. **XOR** deserves special attention: it is the heart of many simple ciphers, because applying the same XOR twice returns the original value. You will meet it again in the Cryptography course. For now, just recognize what AND, OR, and XOR do to bits.

7. How Negative Numbers Are Stored

So far our bytes held 0 to 255, all positive. But programs need negative numbers too. Computers store them with a clever trick called **two's complement**. You do not need the full method yet, but you should know the idea and one consequence.

The idea: for a signed byte, the top bit is used as a sign. If it is 0, the number is positive (0 to 127). If it is 1, the number is negative (−1 to −128). The exact pattern is chosen so that ordinary addition still works without special cases.

The consequence worth remembering: **the same byte can mean different numbers depending on whether it is read as signed or unsigned.** The byte `1111 1111` is 255 if read as an unsigned number, but −1 if read as a signed number. The bits are identical; only the *interpretation* differs. This is exactly the lesson-1 idea, numbers mean nothing until something decides how to read them, and it is the source of a whole family of bugs (integer overflows and sign errors) that you will study in Binary Exploitation. For now: a byte's bits are fixed, but what number they *represent* depends on how the program reads them.

8. Reading a Hex Dump

Putting it together: a **hex dump** is a view of raw bytes, shown in hex, that security people use constantly, to inspect files, memory, and network captures. Almost every tool can produce one. Here is the text `Hi!`, dumped with the `xxd` tool:

```
$ printf 'Hi!' | xxd
00000000: 4869 21                                  Hi!
```

Read it in three columns:

- `00000000` on the left is the **offset**: how far into the data we are, in hex. Here, the start (byte 0).
- `4869 21` in the middle is the **bytes themselves, in hex**. `48`, `69`, `21`, three bytes for three characters.
- `Hi!` on the right is those same bytes shown as text, where they are printable.

So the letter `H` is the byte `0x48`, `i` is `0x69`, and `!` is `0x21`. (Why those particular numbers? That is text encoding, the subject of the next lesson.) Here is a four-byte example:

```
$ printf 'Mina' | xxd
00000000: 4d69 6e61                                Mina
```

`M` = `0x4d`, `i` = `0x69`, `n` = `0x6e`, `a` = `0x61`. Being able to read a dump like this, offset, hex bytes, and text, is a core skill. A reverse engineer reads program bytes this way, a forensic analyst reads disk and memory this way, and a network analyst reads captured packets this way. You now have the alphabet for all of it. The next lesson explains *why* `H` is `0x48`: how text becomes numbers.

Common Mistakes

1. Confusing bits and bytes. A byte is 8 bits. Storage is in bytes (MB), network speed is in bits (Mbps). Divide network speed by 8 to compare.

2. Thinking hex is a different kind of number. Hex is just a short, exact way to *write* the same bytes. One byte is always two hex digits.

3. Trying to memorize conversions. Understand what the columns mean; let a tool do the arithmetic. Nobody converts long binary by hand in real work.

4. Forgetting that a byte's meaning depends on interpretation. `1111 1111` is 255 unsigned or −1 signed. The bits are the same; the reading differs.

5. Miscounting offsets in a hex dump. The offset on the left is in hex, and counts from 0. The byte at offset `0x10` is the 17th byte, not the 10th.

Summary

Term	Remember
Bit / byte	One switch (0/1) / 8 bits, 256 values (0–255)
Binary	Base 2; columns double: 1, 2, 4, 8, 16, …
Hex	Base 16; one digit = 4 bits, two digits = one byte; 0x prefix
Units	KB/MB/GB in bytes (capital B); network speed in bits (lowercase b)
Bitwise	AND (both), OR (either), XOR (different), NOT (flip)
Signed vs unsigned	Same bits can mean different numbers (e.g. 255 vs −1)
Hex dump	Offset · bytes in hex · text; the everyday view of raw data

Check Your Understanding

1. How many bits are in a byte, and how many different values can one byte hold?
2. Convert binary `0010 0110` to decimal. (Hint: use the column values.)
3. Why do security tools show bytes in hex instead of binary?
4. How many bits is one hex digit? How many hex digits is one byte?
5. Your internet is "80 Mbps". Roughly how many megabytes per second is that, and why?
6. What does XOR do to two bits, and why is it useful in simple ciphers?
7. In the hex dump `00000000: 4d69 6e61 Mina`, what byte (in hex) is the letter `n`?

Answers

1. 8 bits; 256 different values (0 to 255).
2. The 1s are in the 32, 4, and 2 columns: 32 + 4 + 2 = 38.
3. Because binary is long and easy to miscount (8 characters per byte), while hex is short and exact: two hex digits per byte, with one digit per 4 bits.
4. One hex digit is 4 bits (a nibble); one byte is two hex digits.
5. About 10 MB/s. Network speed is measured in bits and files in bytes, so you divide by 8: 80 ÷ 8 = 10.
6. XOR gives 1 when the two bits differ. It's useful because applying the same XOR value twice returns the original, so the same key both scrambles and unscrambles.
7. `0x6e` (shown as `6e` in the dump). `n` is the third byte: `4d 69 6e 61` → M, i, n, a.

Practice

1. By hand, convert the decimal number **42** to an 8-bit binary number, using the column values (128, 64, 32, 16, 8, 4, 2, 1). Then convert it to two hex digits.
2. Line up `0b1011` and `0b0110`. Work out, by hand, the result of AND, OR, and XOR on them.
3. If you have a terminal, run `printf 'Den' | xxd` and read the dump: what are the three bytes, in hex? (If you cannot run it, reason about what the columns would show.)

A Worked Example

1. 42: the biggest column that fits is 32 (leaving 10), then 8 (leaving 2), then 2 (leaving 0). So the 32, 8, and 2 columns are 1: `0010 1010`. In hex, split into two nibbles: `0010` = 2 and `1010` = A, so `0x2A`.
2. `1011` AND `0110` = `0010` (only the 2-column is 1 in both). `1011` OR `0110` = `1111`. `1011` XOR `0110` = `1101` (1 where they differ).
3. `printf 'Den' | xxd` gives `44 65 6e`: `D` = `0x44`, `e` = `0x65`, `n` = `0x6e`.

3

Section 3

Representing Text: ASCII, Unicode, and Base64

IN THIS SECTION

1. How Text Becomes Numbers 19

2. ASCII 20

3. The Problem of Other Languages 20

4. Unicode and UTF-8 21

5. Khmer and Other Scripts in Unicode 21

6. Base64: Encoding, Not Encryption 22

7. URL and Percent Encoding 23

8. Why Encoding Matters in Security 23

What You Will Learn

- How text becomes numbers.
- ASCII: the original text encoding.
- Why ASCII was not enough, and how Unicode fixes it.
- UTF-8, and how Khmer and other scripts are stored.
- Base64: encoding that is often mistaken for encryption.
- URL (percent) encoding.
- Why encoding matters in security.

Before You Start

Read lessons 1 and 2. You know everything is bytes, and you can read a hex dump. This lesson answers the question lesson 2 left open: why is the letter `H` the byte `0x48`? The console examples use Python, `base64`, and `xxd`, shown exactly as they ran.

A key idea for the whole lesson: **encoding is a rule for turning information into bytes, and back.** It is not a secret and not protection; it is an agreement about how to read bytes. Confusing encoding with encryption is one of the most common beginner mistakes in security, and we clear it up here.

---

1. How Text Becomes Numbers

A computer only stores numbers (lesson 1), so to store the letter `H`, it must store a *number* that stands for `H`. For this to work, everyone has to agree on which number means which character. That agreement is a **character encoding**: a table that maps each character to a number (and the bytes that hold it).

Think of it like a codebook both sides share: the writer turns `H` into a number using the table, and the reader turns that number back into `H` using the same table. If the two use *different* tables, the reader gets the wrong characters, which is exactly what "garbled text" is.

So the question "why is `H` the byte `0x48`?" has a simple answer: **because a standard table says so**, and everyone agreed to use it. That table, for the basic English characters, is ASCII.

2. ASCII

ASCII (American Standard Code for Information Interchange) is the original character encoding, from the 1960s. It assigns a number from 0 to 127 to each basic English character: the letters A–Z and a–z, the digits 0–9, punctuation, the space, and some invisible "control" characters.

Because the biggest ASCII value is 127, every ASCII character fits in **7 bits**, and so comfortably in one byte. Some landmark values, read for real:

```
$ python3 -c 'print(ord("H"), hex(ord("H"))); print(ord("A"), ord("a"), ord("0"),
ord(" "))'
72 0x48
65 97 48 32
```

So `H` is 72, which is `0x48`, exactly the byte we saw in the lesson-2 hex dump. A capital `A` is 65, a lowercase `a` is 97, the digit character `0` is 48, and a space is 32.

Two patterns worth noticing, because they come up in real work:

- **The digit character `0` is the number 48, not 0.** The *character* `'0'` and the *number* `0` are different things. This catches people constantly.
- **Uppercase and lowercase differ by 32.** `A` is 65 and `a` is 97; `65 + 32 = 97`. That is why changing case is a simple, fast operation.

ASCII is small, simple, and still the foundation: the first 128 values of every modern encoding are still ASCII. But 128 characters cannot hold the world's writing.

3. The Problem of Other Languages

ASCII was designed for American English. It has no `é`, no `ñ`, no Chinese characters, no Arabic, and crucially for us, **no Khmer**. For decades, each language group invented its own incompatible tables that reused the numbers above 127 for different characters. The result was chaos: text written with one table turned into garbage when read with another, because the same byte meant a different character in each.

This is more than an inconvenience. When the same bytes can be read as different characters depending on the assumed table, you get **ambiguity**, and attackers exploit ambiguity. Different parts of a system interpreting the same bytes differently is the seed of real security bugs (for example, sneaking a dangerous character past a filter that reads the bytes one way while the target reads them another).

The world needed **one** table big enough for every character ever used. That is Unicode.

4. Unicode and UTF-8

Unicode is a single, giant table that aims to give every character in every writing system its own unique number, called a **code point**. There is room for over a million code points; the code point for `H` is still 72 (Unicode kept ASCII at the bottom), while Khmer, emoji, and Chinese characters all have their own code points far above 127.

But a code point is just a number. The computer still has to store it as *bytes*, and here is the clever part. **UTF-8** is the rule, by far the most common, for turning Unicode code points into bytes. Its design is brilliant for two reasons:

- **It is backward-compatible with ASCII.** Any ASCII character (code point 0–127) is stored as the *same single byte* as before. So all existing English text is already valid UTF-8, unchanged.
- **It uses more bytes only when needed.** Characters beyond ASCII use two, three, or four bytes.

See it directly. A plain `A` is one byte; `é` takes two; the Khmer letter `ក` takes three:

```
$ python3 -c 'print("A".encode("utf-8").hex(" ")); print("é".encode("utf-8").hex("
")); print("ក".encode("utf-8").hex(" "))'
41
c3 a9
e1 9e 80
```

This has a practical consequence you must internalize: **one character is not always one byte.** In a hex dump, `é` shows up as two bytes, which do not individually look like text:

```
$ printf 'é' | xxd
00000000: c3a9                                     ..
```

The `..` on the right means "these bytes are not printable on their own". So when you count characters, measure text length, or chop a string, bytes and characters can differ. This matters for Khmer, where each letter is several bytes, and it is a frequent source of bugs and of security issues in text handling. **UTF-8 everywhere** is the modern default, and the right choice.

5. Khmer and Other Scripts in Unicode

Khmer has its own block of code points in Unicode, so Cambodian text is fully supported, and that is why these very lessons can be written in Khmer. A couple of points that matter when you handle Khmer text in security work:

- **Each Khmer character is typically three bytes in UTF-8** (as `ក` showed above: `e1 9e 80`). A Khmer word that looks short on screen is many bytes long underneath.

- **What looks like one character can be several code points.** Khmer uses combining marks (for vowels and signs) that attach to a base consonant, so one visible cluster on screen may be built from several code points, and therefore many bytes. The same is true of accented letters and emoji in other scripts.

The takeaway is the same as before: **on-screen appearance, code points, and bytes are three different things.** A forensic analyst searching a disk for a Khmer name, or a developer validating a form, must know that "one character" is not a reliable unit. Always think in the layer you actually need: visible text, code points, or bytes.

6. Base64: Encoding, Not Encryption

Now a different kind of encoding, and the most important security lesson in this chapter. **Base64** is a way to represent *any* binary data (not just text) using only 64 safe, printable characters: A–Z, a–z, 0–9, and two symbols. It exists because some systems (old email, parts of the web, URLs) can only safely carry plain text, so binary data, images, keys, attachments, is Base64-encoded into text to pass through them, then decoded back on the other side.

Here it is, encoding then decoding the same text:

```
$ printf 'Mina Coffee' | base64
TWluYSBDb2ZmZWU=
$ printf 'TWluYSBDb2ZmZWU=' | base64 -d
Mina Coffee
```

You will recognize Base64 on sight after a while: a run of letters and digits, often ending in one or two `=` signs. Now the crucial point:

> **Base64 is encoding, not encryption. It provides no secrecy at all.**

Anyone can decode Base64 instantly, with no key, as the second command shows. It does not hide, protect, or secure anything; it only repackages data into safe characters. Beginners constantly see Base64 in a token or a config file and think the data is "encrypted" or safe. It is not. If you can read the Base64, you (and any attacker) can read the original in one step.

This distinction is central to security:

- **Encoding** (ASCII, UTF-8, Base64, URL-encoding) is a *public rule* for representing data. No secret, no protection. Reversible by anyone.
- **Encryption** (the Cryptography course) uses a *secret key* to make data unreadable without that key.

Whenever you see encoded data, read it; never assume it is protected. And never "secure" a secret by encoding it.

7. URL and Percent Encoding

One more everyday encoding, because you will meet it constantly in web security. Web addresses (URLs) can only use a limited set of characters. Characters outside that set, spaces, and symbols with special meaning (`&`, `=`, `/`, `?`), are replaced by a `%` followed by the character's byte in hex. This is **URL encoding** or **percent encoding**.

```
$ python3 -c 'import urllib.parse; print(urllib.parse.quote("a b&c=/?"))'
a%20b%26c%3D/%3F
```

Read it: the space became `%20` (because a space is byte `0x20`, which is 32, exactly the ASCII space from topic 2), `&` became `%26`, `=` became `%3D`, and `?` became `%3F`. It is the same idea as always, a public rule for packing characters into a restricted channel, using the hex byte values you already know.

URL encoding matters hugely in web security (course 11). Attackers use it to **sneak dangerous characters past filters**: a filter looking for a literal `<` might miss `%3C`, which the browser later turns back into `<`. The same bytes, read at two different stages with two different rules, is the ambiguity theme from topic 3, and it is a real attack technique.

8. Why Encoding Matters in Security

Encoding seems like a dry, technical topic, but it sits under a surprising amount of security:

- **Reading data in the wild.** Tokens, configuration, and captured traffic are full of Base64, hex, and URL-encoding. Recognizing and decoding them is a daily skill (and a whole category of CTF challenges).
- **Encoding is not protection.** Mistaking Base64 or hex for encryption is a classic error that leaves secrets exposed.
- **Ambiguity is an attack surface.** When two parts of a system decode the same bytes differently, attackers slip dangerous input through the gap, injection, filter bypass, and "smuggling" attacks all exploit this.
- **Correct text handling is a security control.** Knowing that one character is not one byte prevents length bugs, truncation bugs, and validation bypasses, especially with non-English text like Khmer.

You now understand how every kind of information, numbers (lesson 2) and text (this lesson), becomes bytes. Next, we leave the world of pure data and look at the machine that works on it: the CPU, memory, and storage.

Common Mistakes

1. Thinking Base64 (or hex) is encryption. It is not. It provides zero secrecy and is reversible by anyone instantly. Never "protect" a secret by encoding it.

2. Confusing the character `'0'` with the number 0. The character `'0'` is ASCII 48. Mixing them up causes real bugs.

3. Assuming one character is one byte. In UTF-8, non-ASCII characters (including every Khmer letter) are several bytes. Counting and chopping text by bytes breaks it.

4. Ignoring which encoding is in use. The same bytes read with the wrong table become garbage, or, worse, become a different character an attacker chose.

5. Trusting a filter that checks only one encoding. Attackers URL-encode or otherwise re-encode dangerous characters to slip past filters that look for the literal form.

Summary

Term	Remember
Character encoding	A public table mapping characters to numbers/bytes
ASCII	The original: basic English in 0–127; `H` = 72 = `0x48`
Unicode	One giant table; every character has a code point
UTF-8	The common rule to store code points as bytes; ASCII-compatible; 1–4 bytes per character
One char ≠ one byte	Non-ASCII (incl. Khmer) take 2–4 bytes; appearance, code points, and bytes differ
Base64	Represents binary as safe text; **encoding, not encryption**; reversible by anyone
URL/percent encoding	`%` + hex byte for special characters; used to bypass filters
Encoding vs encryption	Public rule vs secret key; encoding never protects

Check Your Understanding

1. Why is the letter `H` stored as the byte `0x48`?
2. What is the ASCII value of the digit character `'0'`, and why is that surprising?
3. What does UTF-8 do, and why is it compatible with ASCII?
4. In UTF-8, how many bytes does a typical Khmer letter take? What does that mean for counting characters?
5. A token in a config file is `TWluYSBDb2ZmZWU=`. Is the data secret? What would you do?
6. In a URL you see `%3C`. What character is that, and why might an attacker use it?
7. State the one-sentence difference between encoding and encryption.

Answers

1. Because the ASCII (and Unicode) standard table assigns the character `H` the number 72, which is `0x48` in hex, and everyone agreed to use that table.
2. 48. It's surprising because the character `'0'` is not the number 0; they are different things, and confusing them causes bugs.
3. UTF-8 turns Unicode code points into bytes. It's ASCII-compatible because every ASCII character (0–127) is stored as the same single byte as in ASCII, so existing English text is already valid UTF-8.
4. Typically three bytes. It means one character is not one byte, so counting, measuring, or chopping text by bytes can split or miscount Khmer characters.
5. No, it is not secret, it is Base64, which anyone can decode instantly (it decodes to "Mina Coffee"). You would decode it with `base64 -d` to read it, and never rely on Base64 to hide anything.
6. `%3C` is `<` (byte `0x3C`). An attacker uses it to slip a dangerous character past a filter that only looks for the literal `<`, since the browser decodes it back to `<` later.
7. Encoding is a public, reversible rule for representing data (no secrecy); encryption uses a secret key to make data unreadable without that key.

Practice

1. The word `Hi` is two ASCII characters. Using topic 2's values (`H` = 72 = `0x48`), find the byte for `i` (hint: it comes after `H` in... actually look it up or reason from `a` = 97). Then predict the hex dump of `Hi`.
2. Decide for each: is it **encoding** or **encryption**? (a) Base64; (b) a password-protected ZIP; (c) URL percent-encoding; (d) hex.

3. If you have a terminal: run `printf 'Khmer: ក' | xxd` and count how many bytes the Khmer letter takes versus the English letters.

A Worked Example

1. `i` is lowercase; from the demo, `i` is 105 (you can get it from `ord("i")`), which is `0x69`, matching the lesson-2 dump of `Hi!`. So `Hi` dumps as `48 69`, with `Hi` on the right.
2. (a) encoding; (b) encryption (a key/password makes it unreadable); (c) encoding; (d) encoding. Only the ZIP provides secrecy.
3. The English letters and the space are one byte each; the Khmer letter `ក` is three bytes (`e1 9e 80`), so the single Khmer character takes three times the space of an English one.

4

Section 4

The CPU, Memory, and Storage

IN THIS SECTION

1. The CPU and the Fetch–Execute Cycle 28

2. Registers 29

3. RAM: Fast and Temporary 29

4. Storage: Slower and Permanent 30

5. The Memory Hierarchy and Cache 31

6. Clock Speed, Cores, and Threads 31

7. Why "Volatile" Matters (a Forensics Preview) 32

What You Will Learn

- What the CPU is, and the fetch–execute cycle it repeats.
- What registers are.
- RAM: fast, working memory that forgets when power is lost.
- Storage: slower memory that keeps data without power.
- The memory hierarchy, and why cache exists.
- Clock speed, cores, and threads.
- Why "volatile" memory matters for forensics.

Before You Start

Read lessons 1 to 3. You know everything is bytes. Now we look at the hardware that *works on* those bytes: where they are processed (the CPU) and where they are kept (memory and storage). No code in this lesson; it is about understanding the machine.

---

1. The CPU and the Fetch–Execute Cycle

The **CPU** (Central Processing Unit), or **processor**, is the part of the computer that actually follows instructions. It is the "brain", though as lesson 1 said, it does not think; it obeys. Everything a computer does, the CPU does, by carrying out instructions one after another.

The CPU works by repeating a simple loop, billions of times a second, called the **fetch–execute cycle**:

1. **Fetch:** get the next instruction from memory.
2. **Decode:** work out what that instruction means.
3. **Execute:** do it (add two numbers, move some data, compare two values).
4. Repeat, with the next instruction.

That is the heartbeat of every computer. A program is just a long list of these tiny instructions sitting in memory, and the CPU marches through them, fetch, decode, execute, fetch, decode, execute, faster than you can imagine. One instruction does almost nothing on its own ("add these two numbers"); billions of them per second add up to a running game or a video call.

This loop is the foundation of reverse engineering and exploitation. A reverse engineer reads the very instructions the CPU fetches. An exploit works by changing *which* instruction the CPU fetches next, redirecting that relentless loop to the attacker's own instructions. Hold onto the image of the CPU as a tireless worker reading instructions from memory in order; later courses are about controlling what it reads.

2. Registers

The CPU needs somewhere to hold the few values it is working on *right now*, the two numbers it is about to add, the result, a note of which instruction comes next. For that it has **registers**: a very small number of tiny, extremely fast storage slots built directly into the processor.

Registers are the fastest memory in the computer, because they are inside the CPU itself. But there are very few of them (a few dozen), and each holds only a few bytes. They are a workbench, not a warehouse: just enough room for the job in hand. When the CPU needs a value that is not in a register, it fetches it from the larger, slower memory outside.

A couple of registers are special and worth knowing by name, because they star in later courses:

- The **program counter** (or instruction pointer) holds the address of the *next instruction* to fetch. Change it, and you change what the CPU does next, which is the goal of many exploits.
- The **stack pointer** tracks the top of the stack, a region of memory we meet in lesson 6 and bend in Binary Exploitation.

You do not need the details now. Just know registers exist: tiny, ultra-fast scratch slots inside the CPU, and that one of them decides which instruction runs next.

3. RAM: Fast and Temporary

Registers are tiny, so the CPU needs a much larger place to hold the program it is running and the data it is working on. That place is **RAM** (Random Access Memory), usually just called **memory**.

RAM is where a running program lives: its instructions and its data are loaded into RAM, and the CPU reads and writes RAM constantly as it runs. "Random access" means the CPU can jump straight to any location, by its **address** (a number naming that spot in memory), without reading through everything before it.

RAM has two defining qualities:

- **It is fast** (far faster than a disk), so the CPU is not kept waiting.
- **It is volatile:** when the power goes off, RAM is wiped clean. Everything in it is lost.

That volatility is why you lose unsaved work when the power cuts out: the document was in RAM, not yet written to permanent storage. Volatility is also hugely important in security, which topic 7 returns to: the live secrets of a running system, passwords, keys, decrypted data, are in RAM, and vanish when the machine powers off. This is why investigators capture a suspect's RAM *before* pulling the plug.

4. Storage: Slower and Permanent

RAM forgets when the power goes off, so a computer needs somewhere to keep things permanently: your files, your photos, the operating system itself, everything that must survive a restart. That is **storage** (also called a **drive** or **disk**): the hard drives (HDDs) and solid-state drives (SSDs) in computers, and the flash storage in phones.

Storage is the opposite of RAM in the two qualities that matter:

- **It is slower** than RAM (though modern SSDs are fast, they are still far slower than RAM, and a spinning hard drive is much slower again).
- **It is non-volatile (persistent):** it keeps its data with the power off. That is the whole point of it.

So the normal flow of using a computer is a constant movement between the two: files live in **storage**; when you open one, it is copied into **RAM** to be worked on; when you save, the changes are written back to **storage**. "Save your work" literally means "copy it from volatile RAM to persistent storage".

The RAM/storage split is central to forensics (course 28). **Disk forensics** recovers data from storage, which persists and can even survive deletion (lesson 7). **Memory forensics** captures RAM, which holds live secrets but disappears on power-off. The two are different disciplines precisely because the two kinds of memory behave so differently.

5. The Memory Hierarchy and Cache

We now have three kinds of memory, and they trade off the same way every time: **faster memory is smaller and more expensive; larger memory is slower and cheaper.** You cannot have it all, so computers use several layers at once, called the **memory hierarchy**:

Level	Speed	Size	Keeps data without power?
Registers (in the CPU)	Fastest	A few dozen slots	No
Cache (in/near the CPU)	Very fast	A few megabytes	No
RAM	Fast	Gigabytes	No
Storage (SSD/HDD)	Slow	Hundreds of GB–TB	**Yes**

The idea is to keep what the CPU needs *right now* in the fast, small levels, and everything else in the slow, big levels. The new entry here is **cache**: a small amount of very fast memory, inside or beside the CPU, that holds copies of the data the CPU has used recently or is likely to use next. Because programs tend to reuse the same data and instructions, cache lets the CPU avoid the slower trip to RAM most of the time, which makes a huge difference to speed.

You do not need to manage any of this, the hardware does it automatically. But the mental model, a pyramid from tiny-and-instant at the top to huge-and-slow at the bottom, explains a lot about how computers perform, and it reappears when you study performance and, in exploitation, cache-timing side channels (advanced topics you will meet later).

6. Clock Speed, Cores, and Threads

How fast is a CPU? Three terms describe it, and marketing often blurs them.

- **Clock speed**, measured in **gigahertz (GHz)**, is how many cycles the CPU ticks per second. A 3 GHz CPU ticks three billion times a second, and it does a little work each tick. Higher is faster, all else being equal, but all else is rarely equal.
- **Cores** are complete processors on one chip. A **quad-core** CPU has four CPUs in one, so it can genuinely do four things at the same time. More cores means more real parallel work, which matters a lot for modern multitasking.
- **Threads** are separate streams of instructions. The operating system schedules threads onto cores. A single core can switch between many threads so fast it *looks* simultaneous, and a core may also run two threads at once (a feature often called hyper-threading). We meet threads properly in lesson 6.

The headline: a modern computer does many things *truly* at once (one per core) and many more *apparently* at once (by switching fast between threads). This is why your computer can run a browser, music, and a download together. It also means, in security, that attacks and defenses can run in parallel, and that reasoning about *order* of events across threads is genuinely hard, which is the root of a bug class called **race conditions** (seen in Web Application Security and exploitation).

7. Why "Volatile" Matters (a Forensics Preview)

We end on the single most security-relevant idea in this lesson, because it decides how a real investigation begins. Recall:

- **RAM is volatile:** it holds the live state of a running system, open programs, decrypted files, typed passwords, encryption keys, network connections, but it is **erased the instant the power is cut.**
- **Storage is persistent:** it survives power-off, but a running system's live secrets may exist *only* in RAM and never touch the disk.

This creates a sharp, practical rule in digital forensics, called the **order of volatility**: when investigating a live machine, capture the most volatile evidence *first*. RAM before disk, because RAM vanishes the moment you pull the plug, while the disk will still be there afterward.

Consider a laptop seized while running, with an encrypted disk. The decryption key is in RAM right now, so the data is readable. Pull the plug, and the key is gone forever; the disk is now an unreadable scramble. A trained investigator knows to **capture the RAM first**, while the machine is still on, precisely because of what this lesson taught about volatile versus persistent memory.

You now understand the hardware that works on bytes: the CPU that executes instructions, the registers and cache that feed it, the RAM that holds the running program, and the storage that keeps everything for next time. Next, we look at the software that manages all of this hardware and lets many programs share it: the operating system.

Common Mistakes

1. Thinking the CPU is "smart". It repeats a simple fetch–decode–execute loop, very fast. Its power is speed and literal obedience, not understanding.

2. Confusing RAM with storage. RAM is fast, working, and volatile (lost on power-off). Storage is slower, permanent. "Saving" copies from RAM to storage.

3. Assuming unsaved work or live secrets are on disk. They may be in RAM only. That is why unsaved work vanishes on a power cut, and why investigators capture RAM first.

4. Reading more cores as simply "faster". More cores means more work *in parallel*; clock speed is work *per tick*. They are different, and both matter.

5. Pulling the plug on a live machine during an investigation. It erases RAM, which may hold the only copy of keys and live evidence. Capture volatile memory first.

Summary

Term	Remember
CPU	Repeats fetch → decode → execute, billions of times a second
Registers	Tiny, ultra-fast slots in the CPU; one picks the next instruction
RAM	Fast working memory; holds the running program; **volatile** (lost on power-off)
Storage	Slower, **persistent** memory; keeps files with the power off
Memory hierarchy	Registers → cache → RAM → storage: faster is smaller; cache speeds up RAM
Clock / cores / threads	Ticks per second / parallel processors / streams of instructions
Order of volatility	Capture the most volatile evidence (RAM) first in a live investigation

Check Your Understanding

1. What four steps does the CPU repeat, and what is that loop called?
2. What are registers, and why are they so fast?
3. Give the two defining qualities of RAM, and the two of storage.
4. What does "save your work" actually do, in terms of RAM and storage?
5. What is cache, and why does it speed up a computer?
6. What is the difference between cores and clock speed?
7. A laptop is seized while running, with an encrypted disk. Why must the investigator capture RAM before turning it off?

Answers

1. Fetch, decode, execute (then repeat). It's called the fetch–execute cycle.
2. Tiny, very fast storage slots built into the CPU itself. They're the fastest memory because they're inside the processor, but there are very few and each holds only a few bytes.

3. RAM: fast and volatile (lost on power-off). Storage: slower and persistent (kept with power off).
4. It copies the data from volatile RAM, where the program was working on it, to persistent storage, so it survives a restart.
5. A small amount of very fast memory in or near the CPU that holds recently or soon-needed data, so the CPU avoids the slower trip to RAM most of the time.
6. Cores are complete processors on one chip (more cores = more true parallel work); clock speed is how many cycles the CPU ticks per second (work per tick). Different things.
7. Because the disk's decryption key is in RAM while the machine runs; RAM is volatile, so turning it off erases the key forever, leaving the disk unreadable. The live evidence must be captured first (order of volatility).

Practice

1. Sort these four into the memory hierarchy, fastest/smallest first: RAM, storage (SSD), registers, cache. For each, say whether it keeps data without power.
2. Explain to a non-technical friend, in two sentences, why they lost their unsaved document when the power cut out, using the words RAM and storage.
3. A phone is advertised as "8-core, 3.2 GHz, 8 GB RAM, 256 GB storage". In one sentence each, say what the 8-core, the 3.2 GHz, the 8 GB, and the 256 GB each tell you.

A Worked Example

1. Registers (no) → cache (no) → RAM (no) → storage/SSD (**yes**). Only storage keeps data without power.
2. "Your document was being worked on in RAM, which is fast but temporary and is wiped the instant the power goes off. It had not yet been saved, which would have copied it to storage, the permanent memory that survives a restart."
3. 8-core: it has eight processors, so it can do eight things truly at once. 3.2 GHz: each core ticks 3.2 billion times a second. 8 GB RAM: it can hold 8 gigabytes of running programs and data in fast, volatile working memory. 256 GB storage: it can keep 256 gigabytes of files permanently.

5

Section 5

What an Operating System Does

IN THIS SECTION

1. The OS as the Manager 36

2. The Kernel 37

3. User Space and Kernel Space 37

4. System Calls 38

5. Drivers and Hardware 38

6. Multitasking 39

7. Users, Permissions, and the OS as a Security Boundary 39

What You Will Learn

- What an operating system is, and why every computer needs one.
- The kernel: the core of the OS.
- The split between user space and kernel space.
- System calls: how programs ask the OS for things.
- Drivers, and how the OS talks to hardware.
- Multitasking: how one computer runs many programs.
- Users and permissions, and why the OS is the main security boundary.

Before You Start

Read lessons 1 to 4. You know the hardware: the CPU, memory, and storage. This lesson is about the software that manages all of it, the operating system. No code in this lesson; it builds the mental model that the Linux (03) and Windows (05) courses, and most of the defensive courses, assume.

---

1. The OS as the Manager

Between the bare hardware and the apps you use sits the **operating system** (OS): Windows, macOS, Linux, Android, iOS. From lesson 1's three layers, the OS is the middle one. Its job, in one sentence: **the operating system manages the computer's hardware and resources, and shares them safely among all the programs that want to use them.**

Think of a busy restaurant kitchen. The hardware is the stoves, the fridge, the knives. The programs are the cooks, all wanting to use the same equipment at once. Without a manager, they would collide, grab each other's ingredients, and set the place on fire. The OS is the kitchen manager: it decides who uses what, when, and keeps them out of each other's way.

Concretely, the OS:

- **Shares the CPU** among programs (multitasking, topic 6).
- **Manages memory**, giving each program its own space and keeping them apart.
- **Controls access to storage**, the network, the screen, and other hardware.
- **Enforces the rules** about who is allowed to do what (topic 7).

Every program runs *on top of* the OS and asks it for everything it needs. A program cannot touch the disk or the network directly; it asks the OS. That funnel, every request passing through the OS, is what makes the OS the center of a computer's security.

2. The Kernel

At the heart of the operating system is the **kernel**: the core program that is always running and has complete control over the machine. If the OS is the manager, the kernel is the manager's brain and hands. It is the part that actually talks to the hardware, shares out the CPU and memory, and enforces the rules.

The kernel is special because it runs with **full privilege**: it can do anything to the hardware and to any program. Everything else, your apps, even most of the OS's own tools, runs with less power and must go through the kernel for anything important. This is deliberate. Because the kernel can do anything, it is kept small, carefully written, and protected, and everything else is kept at arm's length from the hardware.

For security, the kernel is the ultimate prize. **If an attacker takes control of the kernel, they control the entire machine**, every program, every file, every secret, with nothing above them to say no. This is why the most serious attacks (and the most powerful malware, like **rootkits**, which you will study in Malware Analysis) aim to reach the kernel, and why defending the kernel's integrity matters so much.

3. User Space and Kernel Space

To protect the kernel, the CPU itself enforces a hard line between two worlds:

- **Kernel space:** where the kernel runs, with full privilege and direct access to all hardware and memory.
- **User space:** where normal programs run, your browser, your editor, with limited privilege and no direct access to hardware.

This is sometimes called the difference between **kernel mode** and **user mode**, and it is enforced by the processor, not just by convention, which makes it a strong boundary. A program in user space simply *cannot* reach out and touch the disk, the network card, or another program's memory directly. The hardware forbids it.

Why build this wall? **Isolation and safety.** If a user-space program crashes or misbehaves, it cannot take down the whole machine or corrupt the kernel, because it never had the power to. A buggy app harms only itself. A malicious app is boxed in. The wall between user space and

kernel space is one of the most important security features in any computer, and crossing it, getting code to run in kernel space, is a major goal in both attack (privilege escalation, course 13) and defense.

4. System Calls

If user-space programs cannot touch hardware directly, how does your editor save a file to disk? It **asks the kernel**, through a **system call** (often shortened to **syscall**).

A system call is the official, controlled doorway from user space into kernel space. When a program needs something only the kernel can do, open a file, read from the network, start another program, allocate memory, it makes a system call: it asks the kernel, "please do this for me." The kernel checks whether the program is allowed, does the work if so, and hands back the result. The program never touches the hardware itself; the kernel does it on the program's behalf.

System calls are the complete list of everything a program can ask the computer to do. That makes them deeply important for security:

- **They are the checkpoints.** Every file opened, every network connection, every new program, goes through a system call, where the kernel can enforce permissions. This is where the OS says yes or no.
- **They are how we watch programs.** Security tools and investigators observe a program's system calls to see what it *really* does, what files it touches, what it sends on the network, regardless of what it claims. Malware analysis (course 29) leans heavily on this: you learn far more from the calls a program makes than from what it says.

So the system-call boundary is both the OS's main control point and a prime place to observe behavior. Keep the picture: a program wanting anything real must knock on the kernel's door with a system call, and the kernel decides.

5. Drivers and Hardware

Computers have endless different hardware: thousands of models of printers, graphics cards, Wi-Fi adapters, webcams. The kernel cannot possibly know the details of every one. So it uses **drivers**: pieces of software, usually part of or loaded into the kernel, that each know how to talk to one specific kind of hardware.

A driver is a translator. The kernel speaks to a driver in general terms ("send this data to the printer"), and the driver turns that into the exact signals that particular printer understands. This is why you sometimes "install drivers" for new hardware: you are giving the kernel the

translator it needs for that device.

Drivers carry a special security weight, because **most drivers run in kernel space**, with full privilege. That makes them a favorite target and tool for attackers: a flaw in a driver is a flaw *in the kernel's world*, often leading straight to full control of the machine. Malicious or vulnerable drivers are a real and serious attack path (studied in exploitation and Windows security). The lesson for now: drivers are powerful, privileged code, and powerful privileged code is always a security concern.

6. Multitasking

Your computer seems to run many programs at once, a browser, music, a chat app, even though, as lesson 4 showed, each CPU core can really only execute one stream of instructions at a time. How? The OS performs **multitasking**: it switches the CPU between programs so quickly that they all appear to run at the same time.

The kernel gives each program a tiny slice of CPU time, a few milliseconds, then pauses it, saves its state (its registers, its place in its instructions), and switches to the next program. This swap is called a **context switch**, and it happens thousands of times a second. With several cores (lesson 4), the computer genuinely runs several programs at once *and* switches among many more on each core. The result is the smooth illusion that everything runs together.

The unit the OS switches between is the **process**, a running program, which is the entire subject of the next lesson. For now, the key security points:

- **The OS decides who runs,** and can stop, pause, or kill any program. That control is a defensive tool (ending a malicious process) and a target (malware tries to keep itself running).
- **Programs are kept apart during switching.** Each process's state is saved and isolated, so one program cannot read another's working data just because they share the CPU. That isolation is, again, a core security guarantee the OS provides.

7. Users, Permissions, and the OS as a Security Boundary

We end where the OS matters most for security. The operating system does not just manage hardware; it enforces **who is allowed to do what.** This builds directly on Security Fundamentals: authentication (who you are) and authorization (what you may do) are enforced, on a real computer, largely by the OS.

Every modern OS has the idea of **users** (accounts), and every file and action has **permissions** that say which users may do it. There is usually a specially powerful account, called **root** (or the **superuser**) on Linux and macOS, and **Administrator** on Windows, which can do almost anything, change the system, read any file, install software. Ordinary user accounts are deliberately limited.

This is the OS applying least privilege (Security Fundamentals, lesson 7) in practice:

- A normal user can run their own programs and edit their own files, but not change the system or read other users' private data.
- Only the powerful account can make system-wide changes, which is why malware and attackers work so hard to **escalate privileges**, to go from an ordinary account to root or Administrator. That jump is the single most common goal after breaking in, and a whole part of the pentesting courses (13) is about it.

So the OS is **the main security boundary on a computer.** It isolates programs from each other (user space vs kernel space), controls every real action (system calls), and enforces who may do what (users and permissions). Nearly every attack is, in some form, an attempt to break one of these OS-enforced boundaries, and nearly every defense is about keeping them intact. That is why the OS gets its own deep courses (Linux and Windows) and why understanding it here is essential.

You now understand the hardware (lesson 4) and the OS that manages it (this lesson). Next, we zoom into the single most important thing the OS manages, and the place where programs actually live and run: processes and their memory.

Common Mistakes

1. Thinking apps talk to hardware directly. They do not. User-space programs ask the kernel, through system calls, for anything real. The kernel is the gatekeeper.

2. Underestimating the kernel. Control the kernel and you control the whole machine. That is why it is small, protected, and the top prize for attackers.

3. Ignoring the user/kernel space boundary. It is enforced by the CPU itself, and crossing it is a major attack goal (privilege escalation) and defense concern.

4. Treating drivers as harmless. Most run in kernel space with full privilege, so a flawed driver is a flaw in the kernel's world, a serious attack path.

5. Forgetting that privilege escalation is the usual next step. After getting in as an ordinary user, attackers try to become root/Administrator. Guarding that jump is central to defense.

Summary

Term	Remember
Operating system	Manages hardware and shares it safely among programs; the middle layer
Kernel	The core, always-running, fully privileged part; control it and you control the machine
User vs kernel space	Limited programs vs the privileged kernel; a hard, CPU-enforced boundary
System call	The controlled doorway programs use to ask the kernel for anything real; a checkpoint and a watch point
Driver	Privileged software that lets the kernel talk to specific hardware; a serious attack surface
Multitasking	The OS switches the CPU among programs fast (context switches), keeping them apart
Users & permissions	The OS enforces who may do what; root/Administrator is all-powerful; attackers escalate to it

Check Your Understanding

1. In one sentence, what does an operating system do?
2. What is the kernel, and why is it the top prize for an attacker?
3. What is the difference between user space and kernel space, and what enforces it?
4. How does a normal program get the OS to save a file or use the network?
5. Why are drivers a special security concern?
6. What is a context switch, and what illusion does it create?
7. What is privilege escalation, and why do attackers want it?

Answers

1. It manages the computer's hardware and resources and shares them safely among all the programs that want to use them.
2. The core program that is always running with full control of the machine. Controlling it means controlling everything, every program, file, and secret, so it is the top target.
3. User space is where limited, normal programs run with no direct hardware access; kernel space is where the fully privileged kernel runs. The CPU itself enforces the boundary.

4. It makes a system call: it asks the kernel to do it. The kernel checks permissions, does the work, and returns the result.
5. Most drivers run in kernel space with full privilege, so a flaw in a driver is a flaw in the kernel's world, often leading to full control of the machine.
6. The OS pausing one program, saving its state, and switching the CPU to another. Done thousands of times a second, it creates the illusion that many programs run at once.
7. Going from an ordinary, limited account to a powerful one (root/Administrator). Attackers want it because the powerful account can change the system and read any data, so it is the usual goal after breaking in.

Practice

1. Explain, using the kitchen-manager analogy, why programs are not allowed to touch the hardware directly, and what they do instead.
2. Name one action your own computer does that must involve a **system call** (for example, saving a file), and say which part, user space or kernel space, actually carries it out.
3. On your computer, you have a normal user account and there is a powerful admin/root account. Give one thing the admin account can do that your normal account cannot, and explain why that limit is a security feature, not just an annoyance.

A Worked Example

1. In the kitchen, cooks (programs) don't each grab the single oven whenever they like, or they'd collide and cause chaos. They ask the manager (the kernel), who decides who gets it and when. Programs likewise ask the kernel, via system calls, instead of touching hardware directly, so the kernel can keep order and enforce the rules.
2. Saving a document must involve a system call to write to the disk. User space (the app) asks; kernel space (the kernel) actually writes the bytes to storage.
3. The admin account can install system-wide software or change security settings; my normal account cannot. That limit is a security feature because if malware runs under my limited account, it also cannot change the system, so the damage is contained, exactly the least-privilege idea from Security Fundamentals.

6

Section 6

Processes, Threads, and Memory

IN THIS SECTION

1. What a Process Is 44

2. How a Process Is Laid Out in Memory 45

3. The Stack and the Heap 45

4. Threads 46

5. Virtual Memory and Isolation 47

6. How the OS Keeps Processes Apart 47

7. Looking at Running Processes 48

What You Will Learn

- What a process is, and how it differs from a program.
- How a process is laid out in memory.
- The stack and the heap.
- Threads, and how they relate to processes.
- Virtual memory, and how it isolates processes.
- How the OS keeps processes apart.
- How to look at the running processes on a real machine.

Before You Start

Read lessons 4 and 5. You know the CPU runs instructions and the OS manages programs. This lesson zooms into a single running program: a **process**. This is one of the most important lessons in the course, because the memory layout here is the ground the Binary Exploitation (18) and Reverse Engineering (17) courses stand on. The console example uses the `ps` command; the layout shown is the real format, with illustrative values.

---

1. What a Process Is

A **program** is a file sitting on disk: a list of instructions, not doing anything. A **process** is a program that is *running*: loaded into memory, with the CPU executing its instructions. The difference is like a recipe versus actually cooking the dish. One recipe (program) can be cooked many times at once (several processes), which is why you can open two windows of the same app.

When you start a program, the OS creates a process for it:

- It loads the program's instructions from disk into RAM.
- It gives the process its own chunk of memory to work in.
- It assigns a unique number, the **PID** (Process ID), to track it.
- It starts the CPU executing the process's instructions (sharing the CPU with all the other processes, lesson 5).

Every process also has a **parent**: the process that started it, identified by its **PPID** (Parent Process ID). When you launch an app, the process that launched it is its parent, and processes form a family tree back to the first process the OS starts at boot (lesson 9). This parent–child

structure matters in security: investigators and defenders look at *what started what*. A word processor spawning a command shell, or a browser launching an unknown program, is the kind of unusual parent–child relationship that reveals an attack.

2. How a Process Is Laid Out in Memory

When a process runs, the OS gives it a block of memory, and that memory is organized into regions, each with a job. Knowing this map is essential for everything that follows. From the bottom (low addresses) to the top (high addresses), a process's memory holds:

Region	What it holds
Code (text)	The program's instructions themselves
Data	Global variables and constants the program set up
Heap	Memory the program requests while running, grows **upward**
Stack	Temporary data for functions, grows **downward**

Picture it as a tall strip of memory. The program's fixed parts, its instructions (code) and its set-up data, sit at the bottom. Above them, two regions grow *toward each other*: the **heap** grows up from below, and the **stack** grows down from the top. Between them is free space they can grow into.

Why does a security course care about this map? Because attacks live in it. The **code** region is what a reverse engineer reads. The **stack** is where classic buffer-overflow attacks happen. The **heap** is where a different family of attacks happens. Keeping or breaking the boundaries between these regions is the entire game of memory-corruption exploitation (course 18). For now, just fix the picture: a running process is a strip of memory with instructions at the bottom and two regions, stack and heap, growing toward each other.

3. The Stack and the Heap

Two of those regions, the stack and the heap, are where a program keeps the data it works with while running. They behave very differently, and the difference matters.

The stack is for short-lived, orderly data, especially the data of **functions** (a function is a named block of instructions a program can call). Every time a function is called, a chunk called a **stack frame** is added to the top of the stack, holding that function's local variables and, crucially, the **return address**: where to continue when the function finishes. When the function returns, its frame is removed. The stack grows and shrinks neatly, last-in-first-out, like a stack of plates.

That return address on the stack is one of the most important facts in all of security. A **buffer overflow** attack works by writing *past* the end of a buffer on the stack until it overwrites the return address, so that when the function finishes, the CPU jumps to an address the *attacker* chose, running the attacker's instructions. Everything in this topic, functions, frames, the return address, is the setup for understanding that attack in lesson 11 and in Binary Exploitation.

The heap is for longer-lived data whose size is not known in advance, memory the program explicitly requests while running ("give me room for this image I just loaded"). The heap is more flexible but less orderly than the stack, and the program is responsible for giving memory back when done. Mistakes here (using memory after giving it back, or forgetting to give it back) cause another whole class of bugs and exploits (**use-after-free**, memory leaks), studied later. The short version: **stack = orderly, automatic, function data; heap = flexible, manual, longer-lived data.**

4. Threads

A process can do more than one thing at a time by using **threads**. A thread is a single stream of execution within a process, a thread of instructions being run. A process always has at least one thread (the program running), and it can create more to do several things at once: one thread handling the screen while another downloads a file, for example.

The key distinction:

- **Processes are isolated from each other.** Each has its own separate memory (topics 5–6). One process cannot see into another's memory.
- **Threads within one process share that process's memory.** They are all inside the same memory block, working on the same data.

That sharing makes threads fast and convenient, but it is also a source of difficulty and bugs. Because threads share memory and run at unpredictable, interleaved times (lesson 4's context switching), two threads can collide, both changing the same data at once, leading to **race conditions**: bugs where the outcome depends on the exact timing of threads. Race conditions are not just reliability bugs; they are a real security vulnerability class (for example, checking a permission in one thread while another changes the thing being checked). You will meet them in Web Application Security and exploitation. For now: a thread is a stream of execution inside a process; threads share memory, which is powerful and dangerous.

5. Virtual Memory and Isolation

Here is a question that reveals something deep: if every process is just put into RAM, what stops one process from reading or trashing another's memory, or the kernel's? The answer is **virtual memory**, one of the most important ideas in how computers work.

The trick: **each process is given its own private view of memory, as if it were the only program on the machine.** A process sees a clean, continuous range of memory addresses starting from zero, called its **virtual address space**, and it believes it owns all of it. But those addresses are not real physical locations. Behind the scenes, the CPU and OS translate each process's virtual addresses into actual physical RAM locations, and they make sure each process's virtual memory maps to a *different*, separate part of physical RAM.

The consequences are profound:

- **Isolation.** Because each process has its own private address space, one process literally cannot name, let alone touch, another process's memory. Address `0x1000` in one process and `0x1000` in another point to completely different physical memory. This is the main thing keeping your banking app's memory safe from a malicious app running beside it.
- **Simplicity for programs.** Every program can be written as if it has the whole machine to itself, and the OS sorts out the real locations.

Virtual memory is why processes are isolated, and that isolation is a core security guarantee, the same theme as lesson 5's user/kernel boundary. When you hear that an attack "broke out of" a process, or read another process's memory, it means they defeated this isolation, which is exactly why doing so is so serious.

6. How the OS Keeps Processes Apart

Virtual memory gives each process its own space; the OS and CPU work together to *enforce* that the walls hold. A few mechanisms, which you only need to recognize:

- **Separate address spaces** (topic 5): each process's memory maps to different physical RAM, so they cannot name each other's data.
- **Permission bits on memory regions.** Each region of a process's memory is marked with what is allowed: readable, writable, executable. For example, the **code** region is executable but not writable (so a program cannot easily rewrite its own instructions), and the **stack** is writable but, on modern systems, *not* executable, a defense (called **NX**, no-execute) specifically designed to stop the classic "run my instructions off the stack" attack. You will meet NX and related defenses by name in Binary Exploitation.

- **The user/kernel boundary** (lesson 5): processes cannot reach the kernel's memory or each other's except through controlled system calls.

The theme across lessons 4, 5, and 6 is now clear: a computer is a set of **boundaries**, process from process, user from kernel, readable from writable from executable, all enforced by the CPU and OS together. Security, at the machine level, is largely the story of keeping those boundaries intact, and attacks are largely the story of breaking them. This lesson has shown you the boundaries around a single process; the exploitation course is about defeating them, and the defense courses about reinforcing them.

7. Looking at Running Processes

All of this is real and visible on any computer. The command that lists running processes is `ps` (on Linux and macOS; Windows has Task Manager and `tasklist`). A typical result looks like this:

```
$ ps -o pid,ppid,%cpu,%mem,comm
  PID  PPID %CPU %MEM COMM
 1521     1   0.3  1.8 WindowServer
 2983  2970   1.2  4.1 Google Chrome
 3001  2983   0.0  2.2 Google Chrome Helper
 4102  3550   0.1  0.4 zsh
```

Read the columns with what you now know:

- **PID**: the process's unique ID.
- **PPID**: its parent's PID, the process that started it. Notice `Google Chrome Helper` (PID 3001) has PPID 2983, which is `Google Chrome`, the browser started its own helper. That parent–child link is topic 1 made visible.
- **%CPU** and **%MEM**: how much processor time and memory the process is using.
- **COMM**: the program's name.

This simple view is the foundation of a lot of defensive work. A SOC analyst or incident responder (courses 25, 27) scans process lists for the abnormal: a process with a strange name, an unusual parent (a document viewer whose child is a command shell), or one eating CPU for no reason. Malware hides among processes, and finding it starts with being able to read a list like this. You will do this hands-on in lesson 12's project, and constantly in the defensive courses.

You now understand a running process inside and out: what it is, how its memory is laid out, the stack and heap, threads, and the isolation that keeps processes apart. Next, we look at where programs and data rest when they are *not* running: files and file systems.

Common Mistakes

1. Confusing a program with a process. A program is a file on disk; a process is that program running in memory. One program can have many processes.

2. Forgetting the return address on the stack. It is the single most targeted value in classic exploitation: overflow a buffer, overwrite it, and redirect the CPU. The whole stack layout matters.

3. Thinking threads are just like separate programs. Threads share their process's memory, which makes them fast but causes race conditions. Processes are isolated; threads are not.

4. Assuming processes can freely read each other's memory. Virtual memory gives each its own private address space; they cannot, unless isolation is defeated, which is a serious attack.

5. Overlooking parent–child relationships. *What started what* is a key signal. An unusual parent (e.g. a document app spawning a shell) is a classic sign of compromise.

Summary

Term	Remember
Program vs process	A file on disk vs that program running in memory
PID / PPID	A process's ID / its parent's ID (what started it)
Memory layout	Code and data at the bottom; heap grows up, stack grows down
Stack	Orderly, automatic function data; holds the **return address** (the exploit target)
Heap	Flexible, manual, longer-lived requested memory
Thread	A stream of execution inside a process; threads **share** memory (→ race conditions)
Virtual memory	Each process gets a private address space → isolation
Keeping apart	Separate address spaces, memory permission bits (NX), the user/kernel boundary

Check Your Understanding

1. What is the difference between a program and a process?
2. What do PID and PPID mean, and why is the parent–child link useful in security?
3. Name the four regions of a process's memory, and say which two grow toward each other.
4. What is stored in a stack frame, and which value in it do buffer-overflow attacks target?

5. How do threads differ from processes in terms of memory, and what bug class does that cause?
6. What is virtual memory, and how does it isolate processes?
7. In a `ps` list, you see a PDF reader as the parent of a command shell. Why might that worry a defender?

Answers

1. A program is a file of instructions on disk, doing nothing; a process is that program running, loaded into memory with the CPU executing it. One program can run as many processes.
2. PID is a process's unique ID; PPID is its parent's PID, the process that started it. The link shows *what started what*, and unusual parent–child relationships reveal attacks.
3. Code (text), data, heap, and stack. The heap (growing up) and the stack (growing down) grow toward each other.
4. A function's local variables and the return address (where to continue after the function finishes). Buffer overflows target the return address, to redirect the CPU.
5. Threads within a process share that process's memory (processes are isolated from each other). Sharing causes race conditions, bugs where the outcome depends on thread timing.
6. Giving each process its own private view of memory (a virtual address space) that the CPU/OS translate to different physical RAM. Because addresses are private and map to separate RAM, one process cannot name or touch another's memory.
7. Because a PDF reader does not normally launch a command shell; that unusual parent–child relationship is a classic sign that the document exploited the reader to run commands.

Practice

1. Explain, with the recipe/cooking analogy, why you can have two windows of the same app open at once.
2. Draw (or describe in words) the memory layout of a process: put code, data, heap, and stack in order, and show which way the heap and stack grow.
3. If you have a terminal, run `ps -o pid,ppid,comm` (Linux/macOS) or open Task Manager (Windows). Find one process and its parent. Does the parent–child relationship make sense (e.g. your terminal is the parent of a command you ran)?

A Worked Example

1. The program is the recipe, a fixed set of instructions on disk. Running it creates a process, like actually cooking the dish. You can cook the same recipe twice at once, two separate pots, so you can run the same app as two separate processes, one per window.
2. From bottom (low addresses) to top (high): **Code** (instructions), then **Data** (globals/constants), then the **Heap** growing **upward**, then free space, then the **Stack** at the top growing **downward** toward the heap.
3. For example, running `ps` from a terminal shows `ps` with its PPID pointing at the shell (`zsh` or `bash`) you typed it in, the shell is the parent that started `ps`. That makes sense: you launched it from the shell.

7

Section 7

Files and File Systems

IN THIS SECTION

1. What a File Really Is 53

2. File Systems 54

3. Paths and Directories 54

4. Metadata and Permissions 55

5. File Types and Magic Bytes, and Why the Extension Can Lie 55

6. Deleting vs. Really Deleting 56

7. Mounting and Devices 57

What You Will Learn

- What a file really is.
- What a file system is, and what it does.
- Paths and directories.
- File metadata and permissions.
- File types and magic bytes, and why the extension can lie.
- The difference between deleting and *really* deleting a file.
- Mounting, and how devices appear as part of the file system.

Before You Start

Read lessons 2, 4, and 6. You can read a hex dump, you know storage is persistent, and you know processes work in memory. This lesson is about how data rests on disk when a program is *not* running. The console examples use `file`, `xxd`, and `ls`, shown from real runs (with a neutral owner name).

---

1. What a File Really Is

You think of a file as a document, a photo, a song. Underneath, a **file is just a named sequence of bytes stored on a drive.** That is the whole definition. A photo, a program, a video, and a text note are all the same kind of thing to the computer: a string of bytes with a name. What makes one a photo and another a program is only *how the bytes are arranged and how they are meant to be read*, which is the lesson-1 idea (numbers mean nothing until something decides how to read them) applied to storage.

This simple truth has real consequences:

- **Any file can be examined as raw bytes** with a hex dump (lesson 2), regardless of what kind of file it is. That is how reverse engineers and forensic analysts look *inside* files.
- **The "type" of a file is not a deep property of the bytes;** it is a convention about how to read them. We will see in topic 5 that the type can be faked.

So when you handle a file in security, remember what it actually is: bytes with a name. Everything else, "it's a PDF", "it's an image", is interpretation.

2. File Systems

A drive (lesson 4) is just an enormous space for bytes. Something has to organize that space, keep track of which bytes belong to which file, where each file starts, what it is called, and who may read it. That organizer is the **file system**: the method the OS uses to store and find files on a drive.

You have met file system names even if you did not know it: **NTFS** (Windows), **APFS** (modern macOS), **ext4** (Linux), **FAT32** and **exFAT** (USB sticks and SD cards). Each is a different scheme for laying out files on storage, but they all do the same core jobs:

- Keep a **directory** (an index) of every file: its name, where its bytes are, and its size.
- Track which parts of the drive are **used** and which are **free**.
- Store each file's **metadata**: its permissions, timestamps, and more (topic 4).

The crucial thing to understand is that **a file's name and its bytes are stored separately.** The file system keeps a record ("the file `menu.txt` lives in these locations on the drive") apart from the actual bytes. This separation seems like a detail, but it is the key to understanding deletion (topic 6) and much of disk forensics. For now: the file system is the drive's filing clerk, keeping an index of where everything is.

3. Paths and Directories

Files are organized into **directories** (also called **folders**), and directories can contain other directories, forming a tree. To name a file's exact location in that tree, you write a **path**: the sequence of directories leading to it, separated by slashes.

- On Linux and macOS: `/home/user/documents/menu.txt` (forward slashes, starting from the root `/`).
- On Windows: `C:\Users\user\Documents\menu.txt` (backslashes, starting from a drive letter).

A path starting from the very top (the **root**) is an **absolute path**, it works from anywhere. A path starting from where you currently are is a **relative path**. Two special names appear in relative paths: `.` means "the current directory" and `..` means "the parent directory, one level up."

That `..` is small but important in security. Because `..` means "go up a level," a carelessly handled path can be tricked into climbing *out* of where it should stay. An attacker who supplies a filename like `../../etc/passwd` may escape the intended folder and reach sensitive files. This is a real and common web attack called **path traversal** (or directory traversal), studied in Web Application Security. The lesson here: paths are not just locations, they are untrusted input when they come from a user, and `..` is the classic trick.

4. Metadata and Permissions

Besides its bytes, every file carries **metadata**: data *about* the file, kept by the file system. Metadata includes the file's size, its timestamps (when it was created, last modified, last accessed), and its **permissions** (who may do what with it).

Permissions are where this connects straight back to Security Fundamentals and lesson 5. On Linux and macOS, every file has permissions for three kinds of people, the **owner**, the **group**, and **everyone else**, and three kinds of action, **read (r)**, **write (w)**, and **execute (x)**. The `ls -l` command shows them:

```
$ ls -l
-rw-r-----  user  staff  menu.txt
-rw-r--r--  user  staff  dot.png
```

Read the first column, `-rw-r-----`, in groups: the first character is the type (`-` for a normal file, `d` for a directory), then three characters for the **owner** (`rw-`: read and write), three for the **group** (`r--`: read only), and three for **everyone else** (`---`: nothing). So `menu.txt` can be read and written by its owner, read by its group, and not touched by anyone else, exactly the least-privilege idea (Security Fundamentals, lesson 7), enforced per file by the OS.

Metadata matters enormously in digital forensics (course 28). Timestamps build a **timeline** of what happened and when (did this file appear right after the suspicious login?). Permissions reveal what an attacker could reach. And because metadata can sometimes be altered, investigators treat it carefully and cross-check it. For now: a file is its bytes *plus* its metadata, and the permissions in that metadata are how the OS controls access to it.

5. File Types and Magic Bytes, and Why the Extension Can Lie

How does a computer know a file is a PNG image and not a text document? You might say "the extension, `.png`." But the extension is just part of the **name**, a hint, and anyone can change it. Rename `photo.png` to `photo.txt` and the bytes do not change at all. So the extension cannot be trusted.

The real answer is usually inside the file. Most file formats begin with a few fixed, recognizable bytes called a **magic number** or **magic bytes**, a signature that identifies the format. A PNG image always starts with the bytes `89 50 4e 47` (the `50 4e 47` spells "PNG" in ASCII, from lesson 3). Look:

```
$ xxd dot.png | head -1
00000000: 8950 4e47 0d0a 1a0a 0000 000d 4948 4452  .PNG........IHDR
```

There it is: `8950 4e47` at the very start, the PNG signature, whatever the file is named. Tools identify files by these signatures, not by the extension. The `file` command does exactly that, reading the content:

```
$ file menu.txt dot.png
menu.txt: ASCII text
dot.png:  PNG image data, 1 x 1, 8-bit gray+alpha, non-interlaced
```

Notice `file` correctly calls `dot.png` a PNG image by looking at its bytes, not its name. This gap, extension versus real content, is a security goldmine:

- **Attackers hide malware by faking the extension.** A file named `invoice.pdf` might really be a program; a photo-looking file might carry hidden data. Checking the *real* type (by magic bytes) instead of trusting the name is a basic defensive and forensic skill.
- **File-upload filters that trust the extension are bypassable.** If a website only checks that a filename ends in `.jpg`, an attacker uploads a program renamed to `.jpg`. Real checks look at the content (Web Application Security covers this).

So: the extension is a changeable hint; the magic bytes are the evidence. Trust the content, not the name.

6. Deleting vs. Really Deleting

When you delete a file and empty the trash, is it gone? Usually **not**, and understanding why is one of the most important ideas in forensics. Recall from topic 2 that a file's *name/index entry* and its *actual bytes* are stored separately.

When you "delete" a file, the file system normally just **removes the index entry and marks that space as free.** The actual bytes are *not* erased, they sit on the drive exactly as before, until the space happens to be reused for another file. It is like removing a book's card from the library catalogue but leaving the book on the shelf: the catalogue says the shelf is empty, but the book is still physically there until someone puts a new book in its place.

This has two big consequences:

- **Deleted files can often be recovered.** Forensic tools scan the drive for the leftover bytes of files whose index entries are gone, a technique called **file carving** (often using magic bytes from topic 5 to spot where files begin). This is how investigators recover files a suspect "deleted", a core part of digital forensics.
- **Truly erasing data takes extra work.** To actually destroy a file's contents, you must *overwrite* the bytes (or encrypt the whole drive so leftover bytes are meaningless, or

physically destroy the drive). "Secure delete" and "wipe" tools do this. The Data Security course (33) covers secure deletion properly.

So "delete" usually means "forget where it is", not "destroy it". That is good news for an investigator recovering evidence, and a warning for anyone who thinks deleting a sensitive file makes it gone.

7. Mounting and Devices

A final idea that ties storage to the file system. A drive's file system does not appear by magic; the OS has to **mount** it: connect the file system on a device into the overall tree of folders you browse, so its files become reachable at some location.

On Windows, mounting a USB stick makes it appear as a new drive letter like `E:`. On Linux and macOS, a mounted drive appears as a folder somewhere in the single tree (for example under `/Volumes` on macOS or `/mnt` on Linux). Either way, mounting is the act of saying "this device's files are now available here."

Mounting matters in forensics and security in a specific, important way. When an investigator examines a suspect's drive, they mount it **read-only**, so that the act of looking cannot change anything on it, preserving the evidence (and often using a hardware **write blocker** to guarantee it). This connects to the forensic principle you will meet in course 28: never alter the original evidence. More broadly, understanding mounting demystifies where your files "come from": every drive, USB stick, and memory card is a file system that the OS mounts into the tree you see.

You now understand how data lives at rest: a file is bytes plus metadata, organized by a file system, named by a path, typed by its magic bytes, recoverable after deletion, and made available by mounting. Next, we look at a special and important kind of file, the **program**, and how source code becomes something the CPU can run.

Common Mistakes

1. Trusting the file extension. The extension is a changeable part of the name. The real type is in the content (magic bytes). Attackers fake extensions to hide malware.

2. Thinking "deleted" means "gone". Deleting usually just removes the index entry; the bytes remain and can be recovered. Secure deletion requires overwriting or encryption.

3. Forgetting that name and bytes are stored separately. This separation explains both deletion and forensic recovery. It is the key idea of the lesson.

4. Ignoring `..` in user-supplied paths. `..` means "up a level" and can be abused to escape a folder (path traversal). Paths from users are untrusted input.

5. Examining evidence on a writable mount. Looking at a drive can change it (timestamps, etc.). Forensics mounts read-only to preserve the original.

Summary

Term	Remember
File	A named sequence of bytes on a drive; "type" is interpretation
File system	The OS's scheme to store/find files (NTFS, APFS, ext4, FAT32); keeps an index
Name vs bytes	Stored separately, the key to deletion and recovery
Path	A file's location in the tree; `..` = up a level (path-traversal risk)
Metadata & permissions	Data about a file; `rwx` for owner/group/others, OS-enforced access
Magic bytes	A file's real type is in its opening signature, not its extension
Delete vs wipe	Delete removes the index (bytes remain, recoverable); wipe overwrites
Mount	Connecting a device's file system into the tree; forensics mounts read-only

Check Your Understanding

1. What, fundamentally, is a file?
2. What does a file system do, and why does it store a file's name separately from its bytes?
3. In `-rw-r-----`, what can the owner, the group, and everyone else each do?
4. Why can't you trust a file's extension to know its real type? What *can* you trust?
5. When you delete a file and empty the trash, what usually happens to its bytes, and why can it often be recovered?
6. What is `..` in a path, and what attack abuses it?
7. Why does a forensic investigator mount a suspect's drive read-only?

Answers

1. A named sequence of bytes stored on a drive. What makes it a photo or a program is only how those bytes are arranged and read.
2. It organizes a drive's space: it keeps an index of every file (name, location, size), tracks used/free space, and stores metadata. It stores name and bytes separately, which is what makes deletion and forensic recovery work the way they do.

3. Owner: read and write (`rw-`). Group: read only (`r--`). Everyone else: nothing (`---`).
4. Because the extension is just a changeable part of the name; renaming doesn't change the bytes. You can trust the content, especially the magic bytes at the start, which identify the real format (as the `file` command does).
5. The file system usually just removes the index entry and marks the space free; the actual bytes remain until overwritten. So tools can scan the drive and recover the leftover bytes (file carving).
6. `..` means the parent directory, one level up. Path traversal abuses it (e.g. `../../etc/passwd`) to escape the intended folder and reach other files.
7. So that the act of examining the drive cannot change anything on it, preserving the original evidence (never alter the original).

Practice

1. Rename a harmless photo on your computer from `.png` to `.txt` and open it in a text editor. What do you see, and what does this prove about extensions versus bytes? (Then rename it back.)
2. If you have a terminal, run `file` on a few different files (a photo, a text file, a PDF) and see what it reports. Does it match or ignore the extension?
3. Explain to a friend, using the library-catalogue analogy, why a file they "deleted" might still be recoverable, and what they would need to do to really destroy it.

A Worked Example

1. Opening the renamed photo in a text editor shows mostly garbage, with maybe a few readable fragments, because a text editor tries to read image bytes as text. It proves the extension is only a label: the bytes are still a PNG, so changing `.png` to `.txt` changed the name but not the content.
2. `file photo.png` reports "PNG image data", `file notes.txt` reports "ASCII text", `file report.pdf` reports "PDF document", each by reading the content's magic bytes, not by trusting the extension. (Rename one and `file` still reports the true type.)
3. "Deleting a file is like removing its card from the library catalogue but leaving the book on the shelf, the catalogue says the space is free, but the book is still there until a new book replaces it, so someone can still find it. To really destroy it, you'd overwrite the space (a secure-wipe tool) or encrypt the whole drive so the leftover bytes are meaningless."

8

Section 8

How Programs Are Built and Run

IN THIS SECTION

1. From Source Code to a Running Program 61

2. Compiled vs. Interpreted Languages 62

3. The Build Pipeline: Compile and Link 63

4. Machine Code and Assembly 63

5. Libraries: Static and Dynamic 64

6. What an Executable File Is (ELF, PE, Mach-O) 65

7. Why This Matters for Reversing and Malware 66

What You Will Learn

- How source code becomes a running program.
- The difference between compiled and interpreted languages.
- The build pipeline: compiling and linking.
- Machine code and assembly.
- Libraries, static and dynamic.
- What an executable file is (ELF, PE, Mach-O).
- Why all of this matters for reverse engineering and malware analysis.

Before You Start

Read lessons 1, 2, and 6. You know everything is bytes, a program is instructions in memory, and the CPU fetches and executes them. This lesson shows where those instructions come from: how a human-written program becomes bytes the CPU can run. It uses a tiny **C** program, compiled with Clang; C is introduced from zero, just enough to see what compiling does. The outputs are from real runs on this machine.

---

1. From Source Code to a Running Program

People do not write the raw numeric instructions a CPU executes; that would be unbearable. Instead, they write **source code**: instructions in a programming language that humans can read, like C, Python, or JavaScript. Source code is just **text** (lesson 3), a file of characters, and the CPU cannot run text. Something must turn that human-readable text into the numeric machine instructions the CPU understands (lesson 2).

Here is a tiny, complete C program. Do not worry about the exact syntax; read it like English. It prints a line, and adds two numbers:

```c
#include <stdio.h>

int add(int a, int b) {
    return a + b;
}

int main(void) {
    printf("Mina Coffee\n");
    printf("2 lattes = %d coffees\n", add(1, 1));
    return 0;
}
```

That is source code: readable text. On its own, the CPU cannot run it. To actually run, it must be turned into machine instructions. There are two main ways to make that happen, compiling and interpreting, and the difference shapes a lot of security.

2. Compiled vs. Interpreted Languages

There are two strategies for running source code.

- **Compiled languages** (like C, C++, Rust, Go) are translated *ahead of time*, in one step, into a file full of machine code, the **executable**. You run the executable directly; the original source is not needed anymore, and not present on the user's machine. This is like translating a whole book into another language once, then handing out the translation.
- **Interpreted languages** (like Python, JavaScript) are translated *as they run*, line by line, by another program called the **interpreter**. You ship the source code, and the user's interpreter reads and runs it on the spot. This is like a live interpreter translating a speech sentence by sentence.

The difference matters for security in a very practical way:

- A **compiled** program ships as machine code, not readable source. To understand what it does, you must **reverse engineer** it (course 17): read the machine instructions. This is the world of analyzing a suspicious `.exe`, where the source is nowhere to be found.
- An **interpreted** program ships as source (or something close to it), so its logic is often much easier to read, sometimes you can just open it. Malicious scripts (in Python, JavaScript, PowerShell) are common precisely because they are easy to write and deliver, and analyzing them is reading code rather than machine instructions.

Many modern languages blend the two (Java and C# compile to an in-between "bytecode" that a runtime then runs), but the core distinction holds: **compiled code hides its logic in machine code; interpreted code ships its logic as readable source.**

3. The Build Pipeline: Compile and Link

Turning C source into an executable actually takes a few steps, together called **building** the program. Run with Clang, our program builds and runs like this:

```
$ clang -o greet greet.c
$ ./greet
Mina Coffee
2 lattes = 2 coffees
```

That one `clang` command hides two important stages:

1. **Compiling:** the compiler translates your source code into machine code, producing an **object file** of raw instructions for your functions. Each source file is compiled to its own object file.
2. **Linking:** the **linker** combines your object file(s) with any **library** code you used (here, the `printf` function, which you did not write, topic 5) into one finished executable that can actually run.

So "compile" loosely means the whole journey, but precisely, compiling makes the machine code for *your* code, and linking stitches it together with everything else into a runnable file. If you have ever seen a "linker error" versus a "compiler error", this is the difference: the compiler could not understand your code, versus the pieces could not be joined together.

You do not need to run builds yet, that is for the programming courses, but recognize the shape: **source code → compile → object files → link (with libraries) → executable.** The reverse of this journey, taking an executable back toward readable form, is exactly what reverse engineering attempts.

4. Machine Code and Assembly

What does the compiler actually produce? **Machine code**: the raw numeric instructions the CPU fetches and executes (lesson 4). Each instruction is a pattern of bytes that means something specific to the processor, "add these two registers", "jump to this address". Machine code is what the executable file contains, and it is not readable by humans in any practical way, it is just bytes.

Because raw machine-code bytes are unreadable, there is a human-friendly version called **assembly language**: a text form where each machine instruction is written as a short mnemonic, like `add`, `mov` (move), `cmp` (compare), or `jmp` (jump). Assembly is almost a one-to-one translation of machine code, just readable. When a reverse engineer studies a compiled program, the tool (a **disassembler**) turns the machine-code bytes back into assembly, and the analyst reads *that*.

You are not learning assembly here, that is the Reverse Engineering course (17). But understand the ladder:

- **Source code** (C, readable, high level) →
- **Machine code** (bytes, what the CPU runs) ↔
- **Assembly** (the readable text form of machine code).

The compiler goes down the ladder (source → machine code). The reverse engineer climbs partway back up (machine code → assembly), because the original source is gone. This is why reversing is hard and skilled work: you are reconstructing meaning from assembly, without the helpful names and structure the original author had.

5. Libraries: Static and Dynamic

Our tiny program used `printf` to print, but we never wrote `printf`. It came from a **library**: a collection of ready-made code that programs reuse instead of reinventing. Almost every real program leans on many libraries (for math, networking, graphics, encryption), which is why Security Fundamentals called an app "mostly other people's code." Libraries connect to a program in two ways:

- **Static linking:** the library's code is copied *into* the executable at build time. The program is self-contained (bigger, but needs nothing else).
- **Dynamic linking:** the library stays a separate file (a `.dll` on Windows, a `.so` on Linux, a `.dylib` on macOS), and the program loads it at run time. Many programs can share one copy, saving space and allowing the library to be updated independently.

This matters for security in several ways you will meet later:

- **Supply-chain risk** (Security Fundamentals, Secure Development): a vulnerability or a backdoor in a library affects every program that uses it. Much of modern security is tracking and patching library flaws.
- **DLL/shared-library hijacking:** because dynamically linked programs load libraries by name at run time, an attacker who plants a malicious library where the program looks can get their code loaded, a real attack technique.

- **Updating:** a dynamically linked library can be patched once to fix every program using it; a statically linked flaw must be rebuilt into each program.

So when you analyze or defend a program, part of the question is always: what libraries does it use, and are they trustworthy and up to date?

6. What an Executable File Is (ELF, PE, Mach-O)

An executable is not just a naked blob of machine code. It is a **structured file** in a standard format, with the machine code *plus* information the OS needs to run it: where the code goes in memory, which libraries to load, where execution starts, and so on. The format depends on the operating system:

- **ELF** on Linux (and Android).
- **PE** (Portable Executable, the familiar `.exe` and `.dll`) on Windows.
- **Mach-O** on macOS (and iOS).

You can see the format with the `file` command (lesson 7's magic bytes at work, executables have signatures too). Here is our compiled program on this machine, and, tellingly, its readable strings:

```
$ file greet
greet: Mach-O 64-bit executable arm64
$ strings greet | grep -i mina
Mina Coffee
```

Two lessons jump out. First, `file` identifies the executable's format from its content. Second, and importantly for security: even though the program is compiled machine code, the **text strings** inside it (like `Mina Coffee`) are still readable, the `strings` tool pulls them straight out. This is why one of the very first steps in analyzing any unknown program, malware included, is to run `strings` on it: web addresses, messages, file names, and clues often sit in plain sight inside a compiled binary. (This is the exact technique the mobile-security course used to read a secret out of an app.)

Understanding executable formats is the foundation of reverse engineering and malware analysis. A malware analyst looking at a suspicious `.exe` is reading a PE file: its structure, its libraries, its strings, and its machine code. Everything in this lesson is what they rely on.

7. Why This Matters for Reversing and Malware

Pulling the lesson together: the journey from source code to a running program is the exact ground that two major security fields stand on.

- **Reverse engineering** (course 17) runs this journey *backward*. Given only an executable, no source, the analyst uses a disassembler to turn machine code into assembly and reconstruct what the program does. Everything here, compiled vs interpreted, machine code vs assembly, executable formats, libraries, defines what that work involves.
- **Malware analysis** (course 29) applies reversing to malicious programs. Is this `.exe` a trojan? What does it do? The analyst checks its format (PE), its strings, its libraries, and its code, exactly the concepts of this lesson, often starting with the easy wins (`file`, `strings`) before the hard work of reading assembly.

Even on defense, this knowledge is everyday. Knowing that compiled programs hide their logic, that interpreted scripts reveal it, that strings leak clues, and that libraries carry risk, shapes how you analyze any unknown program you find on a system.

You now understand how a program comes to exist and run. Next, we step back to the very beginning of a computer's life, how it goes from a cold power-off state to a running operating system: the boot process.

Common Mistakes

1. Thinking the CPU runs source code. It runs machine code. Source code is text that must be compiled (ahead of time) or interpreted (as it runs) first.

2. Treating "compile" and "link" as the same. Compiling makes machine code for your code; linking joins it with libraries into a runnable executable. Their errors are different.

3. Assuming compiled programs reveal nothing. Strings, libraries, and structure are still visible. `strings` on a binary is a classic, fast first step in analysis.

4. Ignoring libraries. Most of a program is library code. A library's flaw is your program's flaw (supply-chain risk), and libraries can be hijacked.

5. Confusing machine code with assembly. Machine code is the bytes the CPU runs; assembly is its readable text form. Reverse engineers read assembly produced by a disassembler.

Summary

Term	Remember
Source code	Human-readable program text; the CPU can't run it directly
Compiled vs interpreted	Translated ahead of time into an executable vs translated line-by-line as it runs
Compile + link	Compile makes machine code for your code; link joins it with libraries
Machine code / assembly	The bytes the CPU runs / the readable text form of those bytes
Library	Reused code; static (copied in) or dynamic (separate file loaded at run time)
Executable format	ELF (Linux), PE/.exe (Windows), Mach-O (macOS); structured file + machine code
Reversing/malware	Run the journey backward; `strings` and `file` are the fast first steps

Check Your Understanding

1. Why can't the CPU run source code directly?
2. What is the key difference between a compiled and an interpreted language, and how does each affect analysis?
3. What do compiling and linking each do?
4. What is the difference between machine code and assembly, and which does a reverse engineer read?
5. What is the difference between static and dynamic linking, and name one security issue each relates to?
6. Name the executable format on Linux, Windows, and macOS.
7. Why is running `strings` on an unknown compiled program a useful first step?

Answers

1. The CPU runs numeric machine instructions, but source code is human-readable text. It must first be compiled (ahead of time) or interpreted (as it runs) into machine code.
2. A compiled language is translated ahead of time into a machine-code executable (source not shipped), so analysis means reverse engineering the machine code. An interpreted language ships its source and is translated as it runs, so its logic is often readable directly.
3. Compiling translates your source into machine code (object files); linking combines those with library code into one finished, runnable executable.

4. Machine code is the raw numeric instructions the CPU runs; assembly is a readable text form of those instructions, almost one-to-one. A reverse engineer reads assembly (produced by a disassembler).
5. Static linking copies the library into the executable (self-contained); dynamic linking keeps it a separate file loaded at run time (shared, updatable). Static relates to needing to rebuild each program to fix a flaw; dynamic relates to DLL/shared-library hijacking and shared updates.
6. ELF (Linux), PE/.exe (Windows), Mach-O (macOS).
7. Because readable text strings (web addresses, messages, file names) survive compilation and sit in plain sight inside the binary, giving quick clues about what the program does before the harder work of reading its code.

Practice

1. Decide for each whether it is **compiled** or **interpreted**, and what that means for reading its logic: (a) a Windows `.exe`; (b) a Python `.py` script; (c) a JavaScript file on a web page.
2. In your own words, explain the pipeline "source → compile → link → executable" to a friend, using the book-translation analogy.
3. If you can run a terminal with a C compiler, compile the `greet` program from topic 1 and run `strings` on the result. Which readable text from the program can you find inside the binary?

A Worked Example

1. (a) `.exe` is compiled machine code, so reading its logic means reverse engineering it. (b) a `.py` script is interpreted and shipped as source, so you can just open and read it. (c) JavaScript on a web page is interpreted and sent as source, so it is readable (anyone can view it in the browser).
2. "Writing source code is like writing a book in a language the CPU can't read. Compiling translates each chapter into the CPU's language. Linking binds those translated chapters together with the quoted passages from other books (libraries) into one finished volume, the executable, which the CPU can finally read and run."
3. After `clang -o greet greet.c`, running `strings greet` reveals readable text including `Mina Coffee` and the `2 lattes = %d coffees` format line, proving that text strings survive compilation and are easy to extract from a compiled binary.

9

Section 9

The Boot Process

IN THIS SECTION

1. From Power Button to Desktop 70

2. Firmware: BIOS and UEFI 71

3. The Bootloader 71

4. Loading the Kernel 72

5. Init and the First Processes 72

6. Where Malware Hides at Boot 73

7. Secure Boot 74

What You Will Learn

- What happens between pressing the power button and reaching the desktop.
- Firmware: BIOS and UEFI.
- The bootloader.
- How the kernel is loaded.
- Init, and the first processes.
- Where malware hides at boot, and why that is so dangerous.
- Secure Boot, and how it defends the start of the chain.

Before You Start

Read lessons 4, 5, and 6. You know about the CPU, the OS and its kernel, and processes. This lesson explains how a cold, powered-off machine becomes a running operating system, the **boot process**, a chain of steps where each stage starts the next. It matters because attackers who get in early, before the OS, gain enormous and hidden power. No code; one small command shows how long a machine has been running since boot.

---

1. From Power Button to Desktop

When you press the power button, the computer is a lump of hardware with nothing running, RAM is empty (it is volatile, lesson 4), and the operating system is just a file on the disk, not yet loaded. Somehow, in a few seconds, you reach a running desktop. That journey is the **boot process** (from "bootstrapping", pulling yourself up by your own bootstraps, because the computer must start itself from almost nothing).

The whole thing is a **chain**: each stage is small and simple, and its main job is to find, check, and start the *next*, bigger stage. Step by step:

1. **Firmware** wakes up the hardware and finds a drive to boot from.
2. The **bootloader** on that drive is started, and it finds and loads the kernel.
3. The **kernel** (the OS core, lesson 5) takes over and sets up the system.
4. The **init** process starts the first user-space programs, leading to the login screen and desktop.

Each link hands control to the next. And that is exactly why boot matters for security: **whoever controls an early link controls everything that comes after it.** A chain is only as trustworthy as its first link. The rest of this lesson walks the chain, then shows what happens when an attacker inserts themselves into it.

2. Firmware: BIOS and UEFI

The very first code to run is not on the hard drive at all, it is **firmware**: a small, permanent program stored on a chip on the computer's motherboard. Because it is on a chip, it is there the instant power arrives, before any disk is read. (Firmware in general is software baked into hardware; it is a whole security field of its own, the IoT and Hardware course.)

The firmware's job at power-on:

- **Wake and test the hardware** (the CPU, RAM, and essential devices). You may have seen this as a logo or a memory check.
- **Find a bootable drive** (a disk, SSD, or USB) in a configured order.
- **Load and run the first piece of software from that drive**, the bootloader (topic 3).

This firmware has two generations you should know by name. **BIOS** (Basic Input/Output System) is the old standard, decades old and now largely replaced. **UEFI** (Unified Extensible Firmware Interface) is the modern replacement on nearly all current computers: more capable, and importantly, able to perform security checks on what it boots (Secure Boot, topic 7). When people say "enter the BIOS" to change boot settings, they usually mean the UEFI firmware settings today.

Firmware is powerful and runs before the OS, which makes it a prized target. Malware that infects firmware (sometimes called a **bootkit** at the firmware level) runs before the operating system and any antivirus, and can survive reinstalling the OS or even replacing the hard drive, because it does not live on the drive. Such attacks are advanced and rare, but they show the principle: the earlier the attacker is in the chain, the harder they are to see and remove.

3. The Bootloader

The firmware does not know how to start a whole operating system; that is too complex for it. So it hands off to a specialist: the **bootloader**, a small program whose one job is to find the operating system's kernel on the drive, load it into memory, and start it.

Because a computer can have more than one operating system installed, the bootloader is also what sometimes shows a menu ("start Windows, or Linux?") and lets you choose. Common bootloaders include GRUB on Linux and the Windows Boot Manager. Whatever the name, the

role is the same: the bridge between the firmware (which knows the hardware) and the kernel (which knows the OS).

The bootloader is another sensitive link, for the same reason as firmware: it runs **before** the kernel and decides *which* kernel to load. An attacker who tampers with the bootloader can load a modified or malicious kernel, or quietly insert code, before any OS protection exists. This class of attack is called a **bootkit**. It is powerful precisely because it sits below the operating system, so the OS's own defenses, running later, may never see it. Guarding the bootloader's integrity is a key reason Secure Boot exists.

4. Loading the Kernel

The bootloader loads the **kernel** into memory and transfers control to it. From lesson 5, the kernel is the privileged core of the operating system, and now it takes charge of the machine for the rest of the session. Its startup work includes:

- **Initializing the hardware** properly, loading the **drivers** (lesson 5) it needs to talk to the disk, network, graphics, and other devices.
- **Setting up memory management**, including the virtual-memory system (lesson 6) that will isolate every process.
- **Preparing to run programs**, then starting the very first user-space process (topic 5).

This is the moment the real operating system comes alive. Everything you learned about the kernel, system calls, user and kernel space, process isolation, begins here, as the kernel builds the managed, protected environment that all your programs will run inside.

For security, note the handover of trust: the bootloader vouched for the kernel and started it with full privilege. If every earlier link was trustworthy, the kernel that now runs is the genuine one. If an earlier link was compromised, the "kernel" now in charge might be an attacker's version, with total control and nothing above it to object. This is why the integrity of the whole chain, firmware → bootloader → kernel, is treated as one connected security problem.

5. Init and the First Processes

A running kernel manages programs, but at this instant there are none, no login screen, no desktop, nothing in user space. So the kernel starts one special first program, traditionally called **init** (on modern Linux it is usually **systemd**; other systems have their equivalents). This is the ancestor of every other process.

Recall from lesson 6 that every process has a parent (its PPID), forming a family tree. **Init is the root of that tree**: it is the first user-space process, and it starts the next ones, the background services, the login screen, and eventually the programs you open, each of which starts more. Trace any process's parents back far enough and you arrive at init. This is also why init has a famous process ID: **PID 1**.

From here, the system reaches the state you recognize: background services running, a login prompt, and after you log in, your desktop and apps. All of it descends from init, which descended from the kernel, which the bootloader loaded, which the firmware started. The chain is complete, and your computer has "booted".

You can see the result of a successful boot with a simple command, `uptime`, which reports how long the machine has been running since it booted:

```
$ uptime
22:32  up 6 days, 23:47, 1 user, load averages: 1.31 1.43 1.70
```

"up 6 days" means the whole chain, firmware to desktop, completed almost a week ago and the system has run since. Every program on it traces back through that boot.

6. Where Malware Hides at Boot

Now the security payoff. The boot chain is attractive to attackers for one reason above all: **persistence**. A piece of malware wants to survive restarts, if it only lived in RAM, a reboot would wipe it (lesson 4). So malware seeks a foothold in the boot process, so it is loaded automatically every time the machine starts, early, and often invisibly.

The earlier in the chain the malware sits, the more dangerous it is:

Where it hides	Name	Why it is dangerous
Firmware (the chip)	Firmware implant / bootkit	Runs before the OS; survives OS reinstall and even disk replacement
Bootloader	Bootkit	Runs before the kernel; can load a tampered kernel; OS defenses may never see it
Kernel (as a driver)	Rootkit	Runs with full privilege; can hide itself, files, and processes from the OS
Startup programs/services	Autostart persistence	Common and easier to find; runs after boot as a normal (if hidden) program

The deeper ones, bootkits and rootkits, are so serious because they load **before** the tools that would detect them, and can then **lie** to those tools, hiding their own files and processes. This is a central theme of Malware Analysis (course 29) and a reason forensic investigators sometimes examine a disk from a *separate*, trusted machine: if the suspect system's own OS is compromised, you cannot trust what it tells you about itself. The principle from topic 1 returns with full force: control an early link, and you control, and can hide from, everything later.

7. Secure Boot

If the danger is a tampered early link, the defense is to **check each link before trusting it.** That is exactly what **Secure Boot** does. Built into UEFI firmware (topic 2), Secure Boot uses digital signatures (a cryptography idea you will study in course 09) to verify that each stage of the boot is genuine and unmodified before running it.

The idea is a **chain of trust**: the firmware only runs a bootloader that carries a valid, trusted signature; that bootloader only loads a kernel that is properly signed; and so on. If any link has been tampered with, its signature no longer matches, and Secure Boot refuses to continue, stopping a bootkit before it runs. Each link verifies the next, so trust extends from the firmware all the way up to the OS.

Related technologies (such as a **TPM**, a Trusted Platform Module chip, and "measured boot") go further, recording what was loaded so the system can prove it booted in a known-good state. You do not need the details now. The concept to carry forward is the important one: because the boot chain is only as trustworthy as its first link, modern computers try to **verify every link before trusting it**, turning a vulnerable chain into a defended one. This is defense in depth (Security Fundamentals) applied to the very start of a computer's life.

You now understand a computer's full life: from power-on through firmware, bootloader, kernel, and init, to a running system, and why each early step is a security battleground. Next, we look at a technology that lets one physical computer safely run many separate systems at once, the foundation of your future practice lab: virtual machines and containers.

Common Mistakes

1. Thinking the OS is the first thing to run. Firmware runs first, then the bootloader, then the kernel. The OS is loaded late in the chain.

2. Underestimating early-stage malware. Bootkits and firmware implants run before the OS and its antivirus, can survive reinstalls, and can hide from the running system. Earlier = more dangerous.

3. Trusting a compromised machine to report on itself. A rootkit can lie to the OS's own tools. Serious forensics examines the disk from a separate, trusted system.

4. Confusing firmware with the OS. Firmware is on a chip on the motherboard and runs at power-on; the OS is a file on the disk, loaded later.

5. Ignoring Secure Boot as "just an annoyance". It verifies each boot stage's signature to stop tampering early, a real defense, not a formality.

Summary

Term	Remember
Boot process	The chain from power-on to a running OS; each stage starts the next
Firmware (BIOS/UEFI)	On a motherboard chip; runs first; wakes hardware and starts the bootloader
Bootloader	Finds, loads, and starts the kernel; runs before the OS
Kernel load	The kernel takes over, loads drivers, sets up memory, starts init
Init (PID 1)	The first user-space process; ancestor of all others
Boot malware	Bootkits/rootkits run early, survive reboots, and hide; persistence is the goal
Secure Boot	UEFI verifies each stage's signature (chain of trust) before running it

Check Your Understanding

1. What is the boot process, and why is it described as a chain?
2. What is firmware, where is it stored, and what are BIOS and UEFI?
3. What is the bootloader's one main job?
4. What does the kernel set up as it starts, and why does that connect to earlier lessons?
5. What is init, what is its PID, and why is it special?
6. Why is malware that hides in firmware or the bootloader so much more dangerous than an ordinary startup program?
7. What does Secure Boot do, and what is a "chain of trust"?

Answers

1. The sequence of steps from pressing power to a running OS. It's a chain because each stage's job is to find, check, and start the next, bigger stage, so control passes link to link.
2. Firmware is a small permanent program on a chip on the motherboard, running the instant power arrives. BIOS is the old standard; UEFI is the modern replacement that can also verify

what it boots (Secure Boot).

3. To find the operating system's kernel on the drive, load it into memory, and start it.
4. It initializes hardware and loads drivers, sets up memory management including virtual memory, and starts the first user-space process. This is where the kernel, system calls, and process isolation from earlier lessons begin.
5. Init is the first user-space process the kernel starts; its PID is 1; it's special because it's the ancestor of every other process, the root of the process tree.
6. Because it runs before the OS and its antivirus even load, can survive OS reinstalls (and firmware implants survive disk replacement), and can hide itself from the running system's tools. Earlier in the chain means more control and better concealment.
7. Secure Boot (in UEFI) uses digital signatures to verify each boot stage is genuine and unmodified before running it. A chain of trust means each link only runs the next if its signature is valid, so trust extends from firmware up to the OS.

Practice

1. Put these boot stages in order, and say in one phrase what each does: kernel, firmware, init, bootloader.
2. Explain to a friend why malware that infects firmware is harder to remove than a virus that is just an app, using what you know about where each lives.
3. If you have a terminal, run `uptime` (Linux/macOS). How long has your machine been running since it last completed the boot chain?

A Worked Example

1. **Firmware** (wakes the hardware, finds a boot drive) → **bootloader** (finds and loads the kernel) → **kernel** (takes over, sets up the system) → **init** (starts the first user programs, leading to login and desktop).
2. "A normal virus is a file on the disk, so wiping or reinstalling the disk removes it. Firmware malware lives on a chip on the motherboard, not the disk, and runs before the operating system even loads, so reinstalling the OS, or even swapping the hard drive, doesn't touch it, and the OS's antivirus never gets a chance to see it."
3. For example, `uptime` showing "up 6 days" means the machine completed its boot almost a week ago and has run continuously since; every process on it traces back to that boot.

10

Section 10

Virtual Machines and Containers

IN THIS SECTION

1. What Virtualization Is 78

2. Hypervisors and Virtual Machines 79

3. Containers, and How They Differ From VMs 79

4. Why Security Work Relies on Both 80

5. Snapshots and Clean States 81

6. The Limits of Isolation 81

What You Will Learn

- What virtualization is.
- Hypervisors and virtual machines (VMs).
- Containers, and how they differ from VMs.
- Why security work relies on both.
- Snapshots, and resetting to a clean state.
- The limits of isolation.

Before You Start

Read lessons 5 and 6. You know the OS manages hardware, and that isolation (keeping programs apart) is a core security idea. This lesson is about running *whole separate systems* on one physical computer, safely isolated from each other. It is the technology behind the home lab you will build in course 08, and behind modern cloud computing. No code in this lesson; it is concepts, with the hands-on work saved for the lab and cloud courses.

---

1. What Virtualization Is

So far, one computer has run one operating system. **Virtualization** breaks that assumption: it lets one physical computer pretend to be several separate computers, each running its own operating system, all at the same time, and all kept apart from one another.

The trick is the same idea as virtual memory (lesson 6), but one level up. Virtual memory gave each *process* its own private view of memory. Virtualization gives each *operating system* its own private view of a whole *computer*: its own CPU, memory, disk, and network, all faked convincingly enough that the OS inside believes it is running on real, dedicated hardware. In reality, the one physical machine is shared among them.

Why is this so useful, especially in security?

- **Isolation.** Each virtual computer is sealed off from the others and, importantly, from the real host machine. Something that happens inside one, even something destructive, is contained.
- **Safety for experiments.** You can run dangerous or untrusted software (including real malware) inside a virtual computer, study it, and then throw the whole thing away, without risking your real machine.

- **Efficiency.** One powerful physical server can host many virtual servers, which is the foundation of cloud computing (course 34).

This single capability, running isolated, disposable, whole systems, is why virtualization underlies both your learning lab and a huge part of modern computing. The next topics explain the two main forms it takes.

2. Hypervisors and Virtual Machines

The most complete form of virtualization is the **virtual machine (VM)**: a full, simulated computer, running a complete operating system of its own, on top of your real one. On your single laptop, you could run a Windows VM and three different Linux VMs at once, each behaving like a separate PC.

Making this work is a special piece of software called a **hypervisor**: the manager that creates VMs and shares the real hardware among them, deciding which VM gets the CPU, how much memory each has, and so on. It is, in effect, a manager of operating systems, the way an OS is a manager of programs (lesson 5). Common tools you will use include **VirtualBox**, **VMware**, and, on Apple Silicon Macs, **UTM** (all covered in the Home Lab course).

The defining feature of a VM is that it is a **complete, independent computer**:

- It runs its **own full operating system** (its own kernel, lesson 5), separate from the host's.
- It has its **own virtual hardware**: virtual disk, virtual network card, virtual memory.
- It is **strongly isolated** from the host and other VMs by the hypervisor.

That strong isolation and completeness make VMs the standard safe place to handle dangerous software. When a malware analyst opens a real virus, they do it inside a VM: the malware infects the virtual machine, not the analyst's real computer, and afterward the whole VM is discarded. The cost of a VM is weight: because each runs a full operating system, VMs use a lot of memory and disk, and take time to start, like booting a whole computer, because that is exactly what they do.

3. Containers, and How They Differ From VMs

A **container** is a lighter form of isolation. Like a VM, it packages a program in its own isolated environment. Unlike a VM, a container does **not** run its own full operating system. Instead, all the containers on a machine **share the host's single kernel**, while each still gets its own isolated view of the file system, processes, and network.

The difference is best seen side by side:

	Virtual Machine	Container
Runs its own OS kernel?	**Yes**, a full OS each	**No**, shares the host's kernel
Weight	Heavy (GBs, slow to start)	Light (MBs, starts in seconds)
Isolation	Very strong (separate OS)	Strong, but weaker than a VM (shared kernel)
Best for	Running different OSes; strong isolation; malware	Packaging and shipping apps consistently

An analogy: a VM is like a separate house, with its own foundation, plumbing, and electricity (its own OS). A container is like a separate apartment in one building, it has its own locked door and rooms (isolated file system and processes), but it shares the building's foundation and utilities (the host's kernel). The apartment is far cheaper and quicker to set up, but it is less independent than a standalone house.

Containers (the best-known tool is **Docker**) transformed how software is built and deployed, because they make an app and everything it needs into one light, portable package that runs the same everywhere. That power, and its security implications, is a whole course of its own (Container and Kubernetes Security, course 35). The key fact to carry: **containers are lighter and faster than VMs but share the host kernel, which makes their isolation strong but not as strong as a VM's.**

4. Why Security Work Relies on Both

Virtualization is not a side topic in security; it is woven through the whole field. You will use VMs and containers constantly.

- **Your home lab is built on VMs** (course 08). You will run an attacker machine (often Kali Linux) and several vulnerable target machines as VMs on your own computer, on an isolated virtual network. This is what makes it *safe and legal* to practise attacks: everything is your own, contained, and disposable. The "only test what you own" rule (Security Fundamentals, lesson 9) is satisfied by design, because the whole lab is virtual machines you own.
- **Malware is analyzed in VMs** (course 29). You detonate real malicious software inside a disposable VM, watch what it does, and discard it, without endangering a real system.
- **The cloud is virtualization at scale** (course 34). Cloud servers are virtual machines and containers running on providers' hardware; securing them is a major field.
- **Defenders use containers and VMs** to run tools in clean, consistent, isolated environments.

So this lesson is unusually practical: the very next hands-on things you do in this program, building a lab, running tools, examining malware, all rest on virtualization. Understanding what a VM and a container *are*, and how they isolate, is the groundwork for all of it.

5. Snapshots and Clean States

One feature of virtual machines deserves its own mention, because it makes them so valuable for learning and for security: the **snapshot**. A snapshot saves the entire state of a VM, its disk and its memory, at a moment in time, so you can **return to that exact state later**, instantly.

Why this is powerful:

- **Experiment without fear.** Take a snapshot of a clean machine, then try something risky, break things, run malware, make a mess. When you are done, restore the snapshot, and the VM is instantly back to its clean state, as if nothing happened. This "undo for the whole computer" is perfect for learning.
- **Repeatable analysis.** A malware analyst infects a VM, studies the damage, then restores a clean snapshot to analyze the sample again from scratch, or to try a different approach, with an identical starting point every time.
- **Fast reset.** Instead of reinstalling an operating system (slow), you restore a snapshot (seconds).

This is why the Home Lab course will have you snapshot your machines before practising: you can attack and infect them freely, then reset to clean in seconds. The combination of **isolation** (nothing escapes the VM) and **snapshots** (reset to clean instantly) is what makes a virtual lab the ideal, safe playground for security skills.

6. The Limits of Isolation

Virtualization's isolation is strong, but, like every security control (Security Fundamentals, lesson 8), it is **not perfect**, and a good security person knows its limits.

The isolation between a VM (or container) and its host is enforced by software, the hypervisor or the kernel, and software can have flaws. An attack that breaks *out* of a virtual machine or container to reach the host, or another guest, is called a **VM escape** or **container escape**. These are rare and difficult (especially for VMs), and highly prized by attackers precisely because they defeat the isolation everyone relies on. They are a serious topic in cloud and container security.

The practical lessons:

- **Containers isolate less than VMs.** Because containers share the host kernel, a flaw in that kernel can affect all of them, and container escapes are generally more feasible than VM escapes. For handling truly dangerous or untrusted code, a **VM gives stronger isolation** than a container.
- **No isolation is absolute.** Even a VM is not a guarantee. When analyzing the most dangerous malware, professionals add extra layers: a dedicated physical machine, an isolated network, no sensitive data nearby, because defense in depth applies here too. Some malware even tries to *detect* that it is in a VM and behaves differently to evade analysis.
- **Keep the host and hypervisor updated.** The isolation is only as strong as the software enforcing it, so patching the hypervisor and host OS matters.

None of this undermines virtualization; it is still one of the most important security tools you have, and your lab depends on it. It simply means you treat isolation as *strong but not magical*: excellent for everyday safety and learning, and layered with extra caution when the stakes are highest.

You now understand how one computer can safely run many isolated systems, the foundation of your lab, malware analysis, and the cloud. You have come a long way: from bits, through the CPU and OS, to processes, files, programs, boot, and virtualization. The next lesson goes back down to the metal for a closer, hands-on look at memory with a little C, the ideas that the exploitation courses build on. Then the final lesson ties everything together on a real, running system.

Common Mistakes

1. Thinking a VM and a container are the same. A VM runs its own full OS (heavy, very strong isolation); a container shares the host's kernel (light, strong but weaker isolation).

2. Assuming isolation is perfect. VM and container escapes exist. Isolation is strong, not absolute, and must be layered for the most dangerous work.

3. Using a container for truly dangerous code. Containers share the host kernel, so a VM (or more) is the safer choice for real malware.

4. Forgetting snapshots. Snapshotting before risky work lets you reset to clean in seconds. It is the feature that makes a VM lab so forgiving.

5. Not updating the host/hypervisor. The isolation is enforced by that software; an unpatched flaw in it can be escaped. Keep it current.

Summary

Term	Remember
Virtualization	One physical computer runs several isolated systems at once
Virtual machine (VM)	A full simulated computer with its own OS; heavy; very strong isolation
Hypervisor	The software that creates VMs and shares the hardware (VirtualBox, VMware, UTM)
Container	Isolated like a VM but shares the host kernel; light and fast; strong but weaker isolation (Docker)
Why it matters	Your lab, malware analysis, and the cloud all run on virtualization
Snapshot	Saved VM state you can restore instantly; "undo" for a whole machine
Limits of isolation	VM/container escapes exist; isolation is strong, not absolute; layer it and patch it

Check Your Understanding

1. What does virtualization let one physical computer do, and how is it like virtual memory?
2. What is a virtual machine, and what is the hypervisor's job?
3. State the key difference between a VM and a container, and the main trade-off.
4. Why is it safe to analyze real malware inside a VM?
5. What is a snapshot, and why is it so useful for learning and for malware analysis?
6. What is a VM escape, and why is it prized by attackers?
7. For handling the most dangerous untrusted code, would you choose a container or a VM, and why?

Answers

1. It lets one physical computer run several separate systems, each with its own OS, at the same time and isolated from each other. Like virtual memory gives each process a private view of memory, virtualization gives each OS a private view of a whole computer.
2. A VM is a full, simulated computer running its own complete OS on top of the real one. The hypervisor is the software that creates VMs and shares the real hardware among them (deciding CPU, memory, etc.).
3. A VM runs its own full OS kernel; a container shares the host's kernel. Trade-off: VMs are heavier but more strongly isolated; containers are lighter and faster but less isolated.
4. Because the malware infects the isolated, disposable virtual machine, not the real host, and the whole VM can be discarded (or reset from a snapshot) afterward.

5. A saved copy of a VM's entire state (disk and memory) that you can restore instantly. It lets you experiment or infect a machine and then reset to a clean state in seconds, with an identical starting point every time.
6. An attack that breaks out of a VM to reach the host or other VMs. It's prized because it defeats the isolation everyone relies on, and it's rare and difficult.
7. A VM, because it runs its own separate kernel and gives stronger isolation than a container, which shares the host kernel (so a container escape is more feasible).

Practice

1. Using the house-vs-apartment analogy, explain to a friend the difference between a VM and a container, including why one is heavier.
2. You want to safely run a suspicious program you downloaded. Describe the steps you would take with a VM and a snapshot so your real computer stays safe.
3. Your future home lab will have an "attacker" machine and a "target" machine as VMs on one laptop. Explain why this setup lets you practise attacks legally and safely, linking it to the "only test what you own" rule.

A Worked Example

1. "A VM is like a separate house: it has its own foundation, plumbing, and electricity, that's its own operating system, so it's fully independent but heavy to build. A container is like an apartment in a building: it has its own locked door and rooms, isolated files and processes, but it shares the building's foundation and utilities, the host's kernel, so it's much lighter and quicker, but a bit less independent."
2. Create a VM with a fresh OS, take a clean snapshot, disconnect or isolate its network, then run the suspicious program inside the VM and observe it. When done, restore the snapshot to wipe any damage, and the program never touched my real computer.
3. Both machines are virtual machines running on my own laptop, on an isolated virtual network, so everything I attack is something I own and nothing reaches the real internet or anyone else's systems. That satisfies "only test what you own or have written permission to test" by design, because the entire lab is mine and contained, and snapshots let me reset it after each exercise.

11

Section 11

A First Look at C and Memory

IN THIS SECTION

1. Why Security People Learn a Little C 86
2. A Tiny C Program 87
3. Variables Are Places in Memory 88
4. Pointers, Gently 88
5. The Stack and a Buffer 89
6. What a Buffer Overflow Is (a Preview) 90
7. Memory Safety, and Why It Matters 91

What You Will Learn

- Why security people learn a little C.
- A tiny C program, read from zero.
- How variables are really places in memory, with addresses.
- What a pointer is, gently.
- The stack and a buffer.
- What a buffer overflow is, as a preview.
- Memory safety, and why it matters so much in security.

Before You Start

Read lessons 2, 6, and 8. You know bytes and addresses, the stack in a process's memory, and that C is compiled to machine code. This lesson gets hands-on with memory using the smallest possible amount of **C**. The goal is not to teach you C (that is a full course), but to let you *see* memory directly, because this is the foundation the Binary Exploitation course (18) is built on. Every program here is complete, compiled with Clang, and shown with its real output. One address will appear; note that **memory addresses change every time a program runs** (a defense called ASLR, which you will meet later), so yours will differ.

---

1. Why Security People Learn a Little C

C is an old language, but it remains central to security for a specific reason: **C works directly with memory.** Most modern languages (Python, JavaScript, Java) deliberately hide memory from you, they manage it automatically and stop you from touching raw addresses, for safety and convenience. C does the opposite: it lets you see and manipulate memory directly, addresses, bytes, the stack, the heap.

That makes C both powerful and dangerous, and that is exactly why it matters here:

- **The systems you will attack and defend are often written in C (or C++).** Operating system kernels, many core programs, and much embedded and IoT software are C. Understanding them means understanding C's view of memory.
- **The most serious class of vulnerabilities, memory-corruption bugs, lives in C-like languages.** Buffer overflows, use-after-free, and their relatives (the whole Binary

Exploitation course) happen because C lets a program misuse memory. You cannot understand these attacks without seeing how C touches memory.

- **It makes memory concrete.** The stack, addresses, and pointers from earlier lessons stop being abstract once you see them in a few lines of C.

You will not become a C programmer in this lesson, and you do not need to. You need to read a handful of tiny programs and *watch what happens to memory*. That is enough to demystify the attacks later.

2. A Tiny C Program

Here is a complete C program. Read it slowly; the comments explain each part.

```
#include <stdio.h>

int main(void) {
    int price = 2500;
    printf("price is %d\n", price);
    printf("price lives at address %p\n", (void *)&price);
    return 0;
}
```

What each piece means:

- `#include <stdio.h>` brings in the standard library so we can use `printf` (lesson 8's libraries).
- `int main(void)` is the **main function**, where the program starts (like `main` in the lesson-1 idea of a starting point).
- `int price = 2500;` creates a **variable** named `price`, of type `int` (a whole number), and puts 2500 in it.
- `printf(...)` prints. `%d` is replaced by the number; `%p` is replaced by an **address** (topic 3).
- `&price` means "the address *of* `price`", the `&` is the key new idea.

Running it:

```
$ clang -o variables variables.c
$ ./variables
price is 2500
price lives at address 0x16b376408
```

The first line is the value, 2500. The second line is something new and important: the **address** where that value lives in memory, shown in hex (lesson 2). Your address will be different, and will change each run. That second line is the whole point of this lesson: a variable is not just a value, it is a value *stored at a place*, and that place has an address.

3. Variables Are Places in Memory

The program just showed it directly, but the idea deserves its own moment, because everything about memory attacks depends on it.

A variable is a named place in memory that holds a value. When you write `int price = 2500;`, the computer:

1. Sets aside a small region of memory (for an `int`, usually 4 bytes) somewhere, often on the **stack** (lesson 6), since `price` is a local variable inside a function.
2. Gives that region the name `price`, so you can refer to it.
3. Stores the bytes for 2500 there.

So a variable has three things: a **name** (`price`), a **value** (2500), and an **address** (where it lives, like `0x16b376408`). Normally you only think about the name and the value, the language hides the address. C lets you ask for the address with `&`, which is why we could print it.

Why does this matter for security? Because **attacks on memory are about addresses and the bytes at them, not names.** An attacker does not care that a variable is called `price`; they care *where* it is in memory and what is next to it. When you read about an exploit "overwriting" a value or "controlling" an address, this is the world it lives in: named values are really just bytes at addresses, and whoever controls the bytes at the right address controls the program. This lesson makes that concrete; the exploitation course weaponizes it.

4. Pointers, Gently

If a variable has an address, you can store that address in another variable. A variable that holds an address is called a **pointer**. Pointers are famous for confusing beginners, so we take them gently, with a program:

```c
#include <stdio.h>

int main(void) {
    int price = 2500;
    int *p = &price;       // p holds the address of price
    printf("value through pointer: %d\n", *p);
    *p = 3000;             // change price via the pointer
    printf("price is now: %d\n", price);
    return 0;
}
```

Two new bits of notation, and one idea:

- `int *p = &price;` creates a pointer `p` and stores in it the *address of* `price` (the `&` from topic 2). So `p` does not hold 2500; it holds *where 2500 lives*.

- `*p` means "the value *at* the address `p` holds", following the pointer to what it points at. Reading `*p` gives 2500.
- `*p = 3000;` writes 3000 *to the address `p` points at*, which is `price`'s location, so it changes `price` itself.

Running it:

```
$ ./pointers
value through pointer: 2500
price is now: 3000
```

We changed `price` to 3000 *without ever naming `price`* on that line, only by going through its address. That is the essence of a pointer: **a pointer is a way to read and write memory by address.** Think of it like a house address written on a slip of paper: the slip is not the house, but anyone holding the slip can go to the house and change what is inside. Pointers are what make C powerful, and they are also where much goes wrong: a pointer holding a *wrong* address lets a program read or write memory it should not. That is the doorway to the next two topics.

5. The Stack and a Buffer

Recall the stack (lesson 6): the region of memory where functions keep their local variables, growing and shrinking as functions are called and return. Our `price` variable lived on the stack. Now meet a **buffer**: a block of memory set aside to hold a sequence of bytes, such as text. Here is a buffer on the stack, used safely:

```c
#include <stdio.h>
#include <string.h>

int main(void) {
    char name[8];                 // room for 7 letters + end marker
    strncpy(name, "Dara", sizeof(name) - 1);
    name[sizeof(name) - 1] = '\0';
    printf("hello, %s\n", name);
    return 0;
}
```

- `char name[8];` sets aside a buffer of 8 bytes on the stack, room for 7 characters of text plus the special end-marker byte (`'\0'`, the "null terminator") that C uses to mark where text ends.
- `strncpy(name, "Dara", sizeof(name) - 1);` copies the text `"Dara"` into the buffer, but **at most 7 bytes**, deliberately not more than the buffer can hold.

Running it:

```
$ ./buffer
hello, Dara
```

The key word above is **at most 7 bytes**. The buffer is a fixed size, 8 bytes, but the data you copy into it might be any length. A careful program always checks that what it writes *fits*. The whole drama of the next topic is what happens when a program does *not* check, and writes more data than the buffer can hold.

6. What a Buffer Overflow Is (a Preview)

Here is the single most famous class of memory bug, and you now have everything needed to understand it. A **buffer overflow** happens when a program writes **more data into a buffer than the buffer can hold**, so the extra data spills past the end of the buffer and over whatever sits next to it in memory.

Imagine the unsafe version of the previous program, which copies text into the 8-byte buffer *without* limiting the length (shown here only to illustrate the bug, it is exactly what you must never write):

```
char name[8];
strcpy(name, user_input);   // DANGER: no length check!
```

If `user_input` is `"Dara"`, fine, it fits. But if it is 50 characters long, `strcpy` happily writes all 50 bytes, running 42 bytes *past* the end of the 8-byte buffer, overwriting whatever is next on the stack. And recall from lesson 6 what sits near local variables on the stack: the **return address**, the record of where the CPU should go when the function finishes.

This is the attack. By supplying carefully crafted over-long input, an attacker can overflow the buffer and **overwrite the return address** with an address of their choosing. When the function returns, the CPU jumps there, running instructions the attacker picked instead of the real program. Over-long *data* has become *control* of the program. This is the classic stack buffer overflow, and it has been behind countless real-world compromises.

You are not exploiting anything here, that is the Binary Exploitation course (18), done in a lab, on your own targets, under the ethics rule. The point of this preview is that the attack is no longer mysterious: it is a direct consequence of everything you have learned, buffers are fixed-size, the stack holds the return address nearby, C does not stop you writing past a buffer, and whoever controls the bytes at the right address controls the program. Defenses you will meet, like **NX** (lesson 6) and **ASLR** (the changing addresses in this lesson), exist specifically to make this attack harder.

7. Memory Safety, and Why It Matters

Step back and ask: why do buffer overflows exist at all? Because C (and C++) are **not memory-safe**: they let a program read and write memory out of bounds, use a pointer to the wrong place, or use memory after it is freed, without stopping it. The program just does it, and the result is a bug or an exploit. This single property is responsible for a large share of the most serious security vulnerabilities in history.

The response, across the whole industry, comes in two forms, and both matter to you:

- **Defenses that make exploitation harder** on existing C/C++ code: NX (non-executable stack), ASLR (randomized addresses), stack canaries, and more. You saw NX and ASLR named already; Binary Exploitation studies them and the techniques that try to defeat them.
- **Memory-safe languages that prevent the bugs entirely.** Languages like **Rust**, and managed languages like Python, Java, and Go, do not let a program write past a buffer or misuse a pointer, they check, or manage memory for you. This is why there is a strong modern push to write new security-sensitive software in memory-safe languages: it removes an entire class of vulnerabilities at the source.

So the humble buffer overflow connects to huge themes: why some languages are "safer" than others, why old C code is a perennial risk, and why the industry is slowly moving to memory-safe languages. Understanding memory, which you now do, is what lets you see all of this clearly.

You have now looked at memory up close and seen, in a few lines, where the most serious attacks come from. One lesson remains: putting everything from this whole course together on a real, running system, inspecting its processes, files, and bytes with your own hands.

Common Mistakes

1. Thinking a variable is only a value. It is a value stored at an address. Memory attacks care about the address and the bytes, not the name.

2. Being scared of pointers. A pointer is just a variable holding an address, a slip of paper with a house address on it. It lets you read/write memory by address.

3. Assuming data always fits its buffer. A buffer is a fixed size; input can be any length. Not checking the length is the bug behind buffer overflows.

4. Thinking a buffer overflow is just a crash. Overwriting the return address can hand control of the program to an attacker. Over-long data becomes control.

5. Believing all languages are equally safe. C/C++ are not memory-safe and allow these bugs; memory-safe languages (Rust, Python, Java) prevent them. The choice matters.

Summary

Term	Remember
Why C	It works directly with memory; the systems and bugs that matter most are C/C++
Variable	A named place in memory: a name, a value, and an **address** (`&` gets the address)
Pointer	A variable that holds an address; `*p` reads/writes the memory it points at
Buffer	A fixed-size block of memory for a sequence of bytes
Buffer overflow	Writing past a buffer's end, over nearby memory, can overwrite the return address
From data to control	Crafted over-long input can redirect the CPU to the attacker's code
Memory safety	C/C++ allow these bugs; memory-safe languages (Rust, Python, ...) prevent them

Check Your Understanding

1. Why do security people learn a little C, rather than only "safe" languages?
2. What three things does a variable have, and which does C let you access with `&`?
3. In plain words, what is a pointer, and what does `*p` mean?
4. What is a buffer, and why must a program check the length of what it writes into one?
5. In a buffer overflow, what spills where, and which nearby value is the dangerous target?
6. How does an attacker turn an over-long input into control of a program?
7. What does "memory-safe" mean, and name one memory-safe and one memory-unsafe language.

Answers

1. Because the systems they attack and defend (OS kernels, core and IoT software) are often written in C/C++, and the most serious vulnerabilities (buffer overflows and relatives) come from C's direct, unchecked access to memory. You cannot understand those attacks without understanding C's view of memory.
2. A name, a value, and an address. C lets you access the address with `&` (e.g. `&price`).
3. A pointer is a variable that holds an address, where another value lives. `*p` means "the value at the address `p` holds", so it reads (or, when assigned, writes) the memory `p` points

at.

4. A buffer is a fixed-size block of memory for a sequence of bytes. The data copied in can be any length, so the program must check it fits; not checking lets data spill past the buffer (a buffer overflow).
5. The extra data spills past the end of the buffer and over whatever is next to it in memory. The dangerous target is the return address on the stack.
6. By supplying crafted over-long input that overflows the buffer and overwrites the return address with an address they chose, so when the function returns, the CPU jumps to the attacker's instructions.
7. Memory-safe means the language prevents out-of-bounds access and pointer misuse (it checks or manages memory), so these bugs can't happen. Memory-safe: Rust (or Python/Java/Go). Memory-unsafe: C (or C++).

Practice

1. Explain, using the "slip of paper with a house address" analogy, what a pointer is and how `*p = 3000` changes a variable without naming it.
2. A program has `char code[4];` and copies a user-supplied coupon code into it with no length check. Explain what goes wrong if the user enters a 20-character code, and why it is dangerous, not just a bug.
3. If you have a C compiler, compile and run the `variables` program from topic 2 twice. Does the printed address stay the same or change between runs? What defense does that changing address represent?

A Worked Example

1. "A pointer is like a slip of paper with a house's address written on it. The slip isn't the house, but anyone holding it can go to that house and change what's inside. So `p` holds `price`'s address, and `*p = 3000` means 'go to the address on the slip and put 3000 there', which changes `price` itself, without writing the name `price` on that line."
2. The buffer `code` holds only 4 bytes, but 20 characters are copied in, so 16 bytes spill past the end of the buffer onto the stack, over whatever is next, including, eventually, the return address. It's dangerous rather than just a crash because an attacker who controls that input can overwrite the return address and redirect the program to their own code.
3. The address changes between runs (for example `0x16b376408` one time, a different value the next). That changing address is ASLR (Address Space Layout Randomization), a defense that randomizes where things sit in memory so attackers cannot rely on fixed addresses.

12

Section 12

Explore a Running System

IN THIS SECTION

1. The Goal: See the Ideas for Real 95

2. Who Am I, and What Is Running 96

3. Identifying and Reading Files 96

4. How Long Up, and How It Started 97

5. Tying It Together 98

What You Will Learn

- How to see, on a real machine, the ideas from this whole course.
- How to inspect the running processes.
- How to identify files by content and read their raw bytes.
- How to see how long the system has been running since boot.
- How it all ties together into the way security people look at a computer.

Before You Start

Read all of lessons 1 to 11. This final lesson is a **project**: instead of new theory, you use what you have learned to explore a real, running computer with a few simple commands. The commands shown use Linux/macOS tools (`whoami`, `ps`, `file`, `xxd`, `uptime`); Windows has equivalents, noted along the way. Run these on **your own computer** only, the one rule from Security Fundamentals. Everything here is safe: you are *looking*, not changing. The outputs shown are from a real machine; yours will differ, which is exactly the point, you are seeing *your* system.

---

1. The Goal: See the Ideas for Real

You have built a complete mental model of a computer: bits and bytes, text encoding, the CPU, memory and storage, the operating system, processes, files, programs, boot, virtualization, and a first look at raw memory. This lesson makes it real. Everything you have learned is visible on the machine in front of you, with a handful of commands.

This is also how security people actually begin with an unfamiliar system. A defender triaging a possibly-compromised machine, or a forensic analyst examining one, asks the same basic questions you are about to ask:

- **Who am I, and what can I do here?** (users and permissions, lesson 5)
- **What is running?** (processes, lesson 6)
- **What are these files, really?** (files and magic bytes, lesson 7)
- **How long has this been up, and how did it start?** (boot, lesson 9)

By the end, you will have looked at a real system the way a professional does, and connected every observation to a concept from this course. Open a terminal on your own computer and follow along.

2. Who Am I, and What Is Running

Start with the most basic question, the one every attacker and defender asks first on a new system: **who am I, and what can I do?** The `whoami` command answers the first part:

```
$ whoami
user
```

That is your user account (lesson 5). On Linux/macOS, the related `id` command shows your groups, and whether you are the powerful **root** account; on Windows, `whoami` works too. This matters because, as lesson 5 taught, *what you are allowed to do depends on who you are*. A limited user and root see very different possibilities, which is why privilege escalation is an attacker's goal.

Next: **what is running?** From lesson 6, `ps` lists processes:

```
$ ps -o pid,ppid,%cpu,%mem,comm
  PID  PPID %CPU %MEM COMM
 1521     1   0.3  1.8 WindowServer
 2983  2970   1.2  4.1 Google Chrome
 3001  2983   0.0  2.2 Google Chrome Helper
 4102  3550   0.1  0.4 zsh
```

Read it with lesson 6's eyes: each process has a **PID**, a **parent** (**PPID**), and its CPU and memory use. Notice `Google Chrome Helper`'s parent is `Google Chrome`, the parent–child tree in action. On a typical machine there are hundreds of processes:

```
$ ps -e | wc -l
     900
```

Nine hundred processes, all sharing the CPU by the OS's multitasking (lesson 5), each isolated in its own virtual memory (lesson 6). (On Windows, open **Task Manager** to see the same thing.) This process list is where a defender hunts for the abnormal: a strange name, an unexpected parent, a process using too much CPU.

3. Identifying and Reading Files

Now turn to files on disk (lesson 7). First, identify what a file *really* is by its content, not its name, using `file` (which reads the magic bytes, lesson 7):

```
$ file greet
greet: Mach-O 64-bit executable arm64
```

`file` reports the true type by looking inside, here, a compiled executable (lesson 8), in this machine's Mach-O format (ELF on Linux, PE on Windows). It would say this even if the file were misleadingly named, which is exactly why defenders trust content over the extension.

Then read a file's raw bytes with `xxd` (lesson 2's hex dump):

```
$ printf 'Mina\n' | xxd
00000000: 4d69 6e61 0a                             Mina.
```

Every idea from lesson 2 and 3 is here: the **offset** on the left, the **bytes in hex** in the middle (`4d` = `M`, `69` = `i`, `6e` = `n`, `61` = `a`, `0a` = the newline), and the **text** on the right. You are reading memory the way a reverse engineer or forensic analyst does. And from lesson 8, remember that running `strings` on a program pulls the readable text straight out of it, a first step in analyzing anything unknown.

With just `file`, `xxd`, and `strings`, you can already look inside any file: what it is, what bytes it contains, and what readable clues it holds. That is the beginning of file analysis, forensics, and reverse engineering.

4. How Long Up, and How It Started

From lesson 9, the whole system came to life through the boot chain, firmware → bootloader → kernel → init → desktop, and everything running now descends from that boot. See how long ago it completed with `uptime`:

```
$ uptime
22:32  up 6 days, 23:47, 1 user, load averages: 1.31 1.43 1.70
```

"up 6 days" means the boot chain finished almost a week ago and the system has run continuously since. Every one of those hundreds of processes traces its ancestry back through **init** (PID 1, lesson 6 and 9) to that boot. The "1 user" is you (lesson 5); the load averages hint at how busy the CPU has been (lesson 4).

If you wanted to go deeper (and you will, in the Linux and defensive courses), the system keeps **logs** of what happened, including at boot and when programs start, which is where investigators reconstruct a timeline (lesson 7's timestamps, the forensics theme). You do not need those commands yet. The point here is the connection: a single `uptime` line ties back to the entire boot lesson, and reminds you that everything running now rests on a chain of trust from power-on.

5. Tying It Together

Step back and see what you just did. With five simple commands, you examined a real computer across the whole stack this course taught:

Command	What you saw	Lesson
`whoami` / `id`	Who you are; your privileges	5 (OS, users, permissions)
`ps`	The running processes, PIDs, parents, resource use	6 (processes)
`file`	A file's real type from its content	7, 8 (files, executables)
`xxd`	A file's raw bytes: offset, hex, text	2, 3 (binary, encoding)
`uptime`	How long since the boot chain completed	4, 9 (CPU, boot)

This is not a toy exercise; it is the real starting posture of security work. A **SOC analyst** or **incident responder** (courses 25, 27) begins triage with exactly these questions: who am I, what is running, what are these files, how and when did this system start. A **forensic analyst** (course 28) reads bytes and timestamps the same way. A **reverse engineer** (course 17) and **malware analyst** (course 29) use `file`, `xxd`, and `strings` as their first moves on an unknown program. You now share their basic vocabulary and their first instincts.

More importantly, you have proven to yourself that a computer is not magic. Every concept from this course, bits, encoding, the CPU, the OS, processes, memory, files, programs, boot, and virtualization, is real, visible, and inspectable with simple tools. That demystified, confident view of the machine is the true deliverable of this course, and the foundation everything ahead is built on.

Where you go next: the Linux course (03) turns these first commands into real command-line skill; the Networking course (04) adds how machines talk; and from there the paths open up, offensive, defensive, engineering. But whichever way you go, you will carry this lesson's habit: when you meet a new system, *look* at it, who, what, which files, how it started, and connect what you see to how computers actually work. You have finished How Computers Work. You understand the machine. Everything from here builds on that.

Common Mistakes

1. Running these on a machine that is not yours. Even "just looking" must be on your own system (or one you have permission for). The one rule never pauses.

2. Memorizing commands without the concepts. The value is connecting each command's output to how computers work. The commands are windows onto the ideas, not the point themselves.

3. Thinking a long process list is suspicious by itself. Hundreds of processes are normal. The skill is spotting the *abnormal* one, by name, parent, or resource use, against that normal background.

4. Trusting a possibly-compromised machine's own output. As lesson 9 warned, a rootkit can hide from these tools. On a truly suspect machine, professionals also examine it from outside.

5. Stopping here. This is a first look. The Linux and defensive courses turn these first commands into real investigative skill.

Summary

Idea	Remember
The project	Use simple commands to see this whole course on a real machine
`whoami` / `id`	Who you are and what you may do (users and permissions)
`ps`	What is running: PIDs, parents, resources (processes)
`file`	A file's real type, from content not name (files, executables)
`xxd` / `strings`	A file's raw bytes and readable text (binary, encoding, analysis)
`uptime`	How long since the boot chain completed (boot)
The real lesson	A computer is inspectable, not magic; this is how security work begins

Check Your Understanding

1. What four basic questions does a defender (and this project) ask about an unfamiliar system, and which lesson does each connect to?
2. What does `whoami` tell you, and why does it matter what account you are?
3. In a `ps` listing, what would make one process look suspicious among hundreds of normal ones?
4. Why do you trust `file`'s answer over a file's extension?
5. In the dump `4d69 6e61 0a Mina.`, which byte is the letter `n`, and what is `0a`?
6. What does "up 6 days" tell you, and what does every running process trace back to?
7. Why might a professional not fully trust the output of these commands on a truly compromised machine?

Answers

1. Who am I and what can I do (lesson 5, users/permissions); what is running (lesson 6, processes); what are these files really (lesson 7/8, files/executables); how long up and how did it start (lesson 4/9, CPU/boot).
2. Your user account. It matters because what you're allowed to do depends on who you are, a limited user and root have very different powers, which is why attackers try to escalate privilege.
3. A strange or unexpected name, an unusual parent (e.g. a document app whose child is a shell), or unusually high CPU/memory use, anything abnormal against the normal background.
4. Because the extension is just a changeable part of the name, while `file` reads the content's magic bytes, so it reports the true type even if the file is misleadingly named.
5. `6e` is the letter `n` (the third byte: `4d` M, `69` i, `6e` n, `61` a). `0a` is the newline character.
6. That the boot chain completed almost a week ago and the system has run continuously since. Every running process traces back through init (PID 1) to that boot.
7. Because a rootkit (lesson 9) runs with full privilege and can hide its own files and processes from the system's own tools, so the machine can lie about itself; serious work also examines it from outside.

Practice

This is the project. On **your own computer**, open a terminal (or Task Manager on Windows) and do the following, writing a short note connecting each to a lesson:

1. Run `whoami` (and `id` on Linux/macOS). Who are you? Are you an administrator/root or a limited user?
2. Run `ps -e | wc -l` (Linux/macOS) or open Task Manager (Windows). Roughly how many processes are running? Pick one and find its parent, does the relationship make sense?
3. Run `file` on two different files (e.g. a photo and a document), then `xxd` on a small text file. Does `file` match the extension? Can you read the offset, hex, and text columns in the dump?
4. Run `uptime` (Linux/macOS) or check uptime in Task Manager's Performance tab (Windows). How long since your machine booted?

Write one sentence per step saying which course concept it showed you. That write-up is a perfect first entry for the portfolio Security Fundamentals recommended.

A Worked Example

1. `whoami` shows my normal account, and `id` shows I'm not root, I'm a limited user, which (lesson 5) means malware running as me couldn't change the whole system.
2. `ps -e | wc -l` shows about 300 processes. I picked my terminal's shell and found its parent is the terminal app, which makes sense, the terminal launched the shell (lesson 6's process tree).
3. `file photo.png` says "PNG image data" and `file notes.txt` says "ASCII text", matching the content, not just the extension (lesson 7). `xxd notes.txt` shows the offset, the hex bytes, and the text, exactly lesson 2's three columns.
4. `uptime` says "up 2 days", so my machine completed its boot chain two days ago and every process traces back to it (lesson 9).

References

Durable, authoritative sources used to write this book. Re-check before relying on a specific detail, and record the exact tools and versions behind any command output.

Core computing

Operating Systems: Three Easy Pieces (OSTEP), free online — https://pages.cs.wisc.edu/~remzi/OSTEP/

Charles Petzold, *Code: The Hidden Language of Computer Hardware and Software* (book)

Computer Systems: A Programmer's Perspective (book)

Ben Eater, breadboard computer videos — https://eater.net

Numbers, text, encoding

The Unicode Standard — https://www.unicode.org

RFC 4648 (Base64) — https://www.rfc-editor.org/rfc/rfc4648

RFC 3986 (URIs, percent-encoding) — https://www.rfc-editor.org/rfc/rfc3986

OS, programs, boot

The Linux man pages — https://man7.org/linux/man-pages/

LLVM / Clang documentation — https://clang.llvm.org

UEFI Specification — https://uefi.org/specifications

The tools used for the command output in this book (`xxd`, `base64`, `file`, `ps`, `uptime`, `clang`) were run on a macOS machine; output differs by system and version.